

Санкт-Петербургский государственный университет

Программная инженерия

Группа 24.М71-мм

Исследование и применение больших
языковых моделей для автоматизации
построения исполняемых сред
программных проектов

Ван Цзыхань

Отчёт по учебной практике
в форме «Эксперимент»

Научный руководитель:
ст. преподаватель кафедры ИАС, к.ф.-м.н. Азимов Р. Ш.

Санкт-Петербург
2026

Оглавление

Abstract	4
Аннотация	5
Введение	7
Постановка задачи	10
1. Обзор современных технологий и решений	11
1.1. Анализ современных подходов	11
1.2. Docker как основа выполнения и управления средой . . .	15
1.3. Анализ существующих подходов	17
1.4. Выводы	19
2. Архитектура системы сборки	21
2.1. Цели проектирования системы	21
2.2. Общая архитектура системы	22
2.3. Структура графа выполнения	23
2.4. Выбор технологий и архитектурных решений	25
2.5. Выводы по главе	25
3. Модель и механизмы выполнения	27
3.1. Формирование начального контекста	28
3.2. Механизм ReAct-цикла выполнения	30
3.3. Управление поведением: системные промпты	34
3.4. Механизм обеспечения устойчивости выполнения	34
3.5. Механизм генерации Dockerfile	35
3.6. Условия завершения выполнения	36
3.7. Выводы по главе	37
4. Определение состояния и сборка графа	38
4.1. Модель состояния: структура AppState	38
4.2. Инициализация вычисления и внешний контекст выполнения	41

4.3.	Архитектура графа выполнения	42
4.4.	Центральный ReAct-цикл	45
4.5.	Механизм снимков и отката контейнера	51
4.6.	Инструментальный слой	53
4.7.	Терминальные узлы и завершение работы	55
4.8.	Выводы по разделу	57
5.	Экспериментальное исследование	58
5.1.	Условия эксперимента и описание данных	58
5.2.	Описание набора данных	59
5.3.	Исследовательские вопросы	59
5.4.	Метрики оценки	60
5.5.	Сводные результаты эксперимента	60
5.6.	Анализ результатов	62
5.7.	Выводы по главе	66
	Заключение	68
	Список литературы	70

Abstract

This thesis presents LangGraph, an agent-oriented framework for automated repository analysis, test-environment recovery, and Dockerfile synthesis for open-source software projects. Configuring executable environments for unfamiliar repositories remains time-consuming and error-prone due to incomplete or inconsistent project metadata. To address this challenge, LangGraph integrates structured workflow orchestration with large language model reasoning to analyze repository artifacts, infer build and test requirements, iteratively repair missing dependencies, and generate reproducible container specifications. The framework is implemented as a graph-based multi-stage pipeline and features an observation-and-repair loop that leverages test-collection feedback as a control signal to guide environment refinement, enabling the system to distinguish dependency-related failures from internal configuration errors. Evaluated on 98 open-source repositories, LangGraph achieves a 92.8% success rate in fully configuring execution environments and passing test suites, significantly outperforming baseline methods (pipreqs-based: 16.67%, direct LLM generator: 22.1%, SWE-agent: 9.0%), with all improvements statistically significant ($p < 0.001$). Successful configurations require a median of 9 agent iterations and an average setup time of 670 seconds per project. These results demonstrate that execution-feedback-driven agent orchestration can effectively automate reproducible environment setup and substantially reduce manual effort in repository onboarding.

Keywords: automated environment configuration, large language model reasoning, agent-oriented framework, graph-structured workflow, execution-feedback loop, Dockerfile synthesis.

Аннотация

В данной работе представлен LangGraph — агентно-ориентированный фреймворк для автоматизированного анализа репозиторий, восстановления тестовых сред выполнения и синтеза Dockerfile для проектов с открытым исходным кодом. Конфигурирование исполняемой среды для незнакомых репозиторий остаётся трудоёмкой и подверженной ошибкам задачей, что обусловлено неполнотой или несогласованностью метаданных проектов. Для решения этой проблемы LangGraph объединяет структурированную оркестрацию рабочих процессов с механизмами рассуждения на основе больших языковых моделей для анализа артефактов репозитория, вывода требований к сборке и тестированию, итеративного устранения отсутствующих зависимостей и генерации воспроизводимых спецификаций контейнеризации. Фреймворк реализован в виде графо-ориентированного многоэтапного конвейера и включает цикл «наблюдение–исправление», в котором обратная связь на этапе сбора тестов используется как явный управляющий сигнал для уточнения среды выполнения. Данный механизм позволяет различать ошибки, связанные с зависимостями, и внутренние конфигурационные дефекты проекта, повышая надёжность по сравнению с одностадийными подходами генерации. Экспериментальная оценка, проведённая на 98 репозиториях с открытым исходным кодом, показала, что LangGraph достигает уровня успешной конфигурации и прохождения тестов в 92,8%, существенно превосходя базовые методы (на основе `pipreqs` — 16,67%, прямой генерации LLM — 22,1%, SWE-agent — 9,0%); все различия статистически значимы ($p < 0,001$). Для успешной настройки требуется медианное значение в 9 итераций агента при среднем времени выполнения 670 секунд на проект. Полученные результаты демонстрируют, что оркестрация агентов с использованием обратной связи от выполнения эффективно автоматизирует воспроизводимую настройку сред и существенно снижает трудозатраты при первичном освоении репозиторий.

Ключевые слова: автоматизированная конфигурация сред выполнения, большие языковые модели, анализ программных репозиторий,

агентные системы, оркестрация рабочих процессов, генерация Dockerfile.

Введение

В последние годы большие языковые модели (LLM) достигли значительного прогресса в решении задач программной инженерии, таких как генерация кода и автоматический синтез патчей [1]. Системы, такие как SWE-agent [2], OpenHands [3] и Copilot [4], способствуют смене парадигмы в области LLM4SE, где глубокая интеграция моделей в процессы разработки становится доминирующей тенденцией. Однако переход от статической генерации кода к динамическому выполнению задач по-прежнему ограничивается фундаментальной проблемой — отсутствием высококачественных интерактивных сред выполнения для обучения и оценки. Несмотря на высокие результаты LLM в задачах статической генерации кода, их возможности в области автоматизированной динамической верификации остаются ограниченными.

Как отмечают Gao и соавторы [5], реальные программные проекты обладают сложной структурой и специфическими зависимостями, что требует значительных усилий даже со стороны опытных разработчиков для настройки окружения и отладки. Вследствие этого зависимость от воспроизводимых сред выполнения стала критическим препятствием на пути к достижению сквозной автоматизации задач в системах программной инженерии на основе LLM.

Для более глубокого понимания данной проблемы необходимо рассмотреть процесс построения среды выполнения на практике. В типичных рабочих процессах программной инженерии создание исполняемой среды включает несколько взаимосвязанных этапов: разрешение и установку зависимостей, настройку системы сборки, инициализацию среды выполнения и запуск тестов. Этот процесс в значительной степени опирается на метаданные, предоставляемые проектом (например, спецификации зависимостей и сценарии сборки), которые в реальных репозиториях с открытым исходным кодом часто оказываются неполными или противоречивыми, что делает настройку среды сложной и подверженной ошибкам.

Существующие подходы к автоматизированному построению сред

можно условно разделить на две основные парадигмы. Первая основана на эвристических инструментах, таких как `repo2docker` [7] и `pipreqs` [8], которые генерируют `Dockerfile` на основе статических метаданных проекта. Подобные методы являются относительно лёгкими и быстрыми, однако плохо справляются со сложными конфликтами зависимостей, несовместимостью версий и отсутствующими или противоречивыми метаданными, что часто встречается в реальных проектах с открытым исходным кодом.

Вторая парадигма использует агентов на базе больших языковых моделей, взаимодействующих со средой через последовательность действий. Системы, такие как `Repo2Run` [9] и `SWE-agent` [2], демонстрируют, что LLM-агенты способны выполнять построение среды, интерпретируя обратную связь и предпринимая шаги по устранению ошибок. В частности, `Repo2Run` показывает высокие результаты (86,0%) благодаря структурированному конвейеру с двойной средой выполнения и заранее определёнными стратегиями восстановления. Однако управление выполнением в подобных системах по-прежнему в значительной степени основано на неявной интерпретации текстовой обратной связи и не содержит явных механизмов управления потоком выполнения для адаптивного ветвления и итеративного уточнения. В целом существующие подходы либо не обладают явной структурой управления выполнением, либо полагаются на неструктурированную интерпретацию ошибок, что ограничивает их устойчивость в сложных сценариях отказов.

Для решения указанных проблем в данной работе предлагается `LangGraph` — агентно-ориентированный фреймворк, преобразующий традиционный процесс статической конфигурации среды в динамический замкнутый цикл «восприятие—рассуждение—выполнение—адаптация». В отличие от конвейерных систем, `LangGraph` моделирует построение среды как графо-ориентированный процесс с явной поддержкой условных переходов и итераций, управляемых обратной связью. Основу `LangGraph` составляет агентный цикл в стиле `ReAct` [11], функционирующий внутри `LangGraph StateGraph` [10], где обратная связь от выполнения преобразуется в структурированные сигналы принятия решений посредством

многоуровневого механизма классификации кодов завершения. Это позволяет реализовать детерминированную маршрутизацию выполнения, управляемую программно, а не исключительно интерпретацией сообщений об ошибках на естественном языке со стороны LLM.

Основные результаты и вклад данной работы заключаются в следующем:

- предложен замкнутый агентный фреймворк для построения среды выполнения, моделирующий процесс как графо-ориентированную систему с поддержкой ветвления и итеративного уточнения на основе структурированной обратной связи;
- разработана надёжная и обратимая инфраструктура выполнения на базе Docker, включающая механизмы атомарного отката состояний, защиту на уровне команд и принудительное вмешательство при возникновении ошибок выполнения;
- предложен механизм структурированной верификации и абстракции ошибок, преобразующий необработанные ошибки выполнения в формализованные сигналы принятия решений с использованием иерархической классификации кодов завершения;
- проведена экспериментальная оценка на реальных Python-репозиториях GitHub, в ходе которой система достигла успешности 92,8% при полном построении исполняемых сред, существенно превзойдя эвристические методы (16,67%) и существующие LLM-подходы, включая SWE-agent (9,0%) и Repo2Run (86,0%).

Постановка задачи

Целью данной дипломной работы является разработка прототипа системы на основе LLM-агента для автоматического конфигурирования и верификации исполняемых сред программных проектов в Docker-контейнерах по данным репозитория исходного кода.

Под исполняемой средой в работе понимается контейнеризованная программная среда, содержащая зависимости и настройки, необходимые для запуска тестов и воспроизведения проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ средств автоматизации программных окружений и рассмотреть существующие подходы к построению воспроизводимых сред, а также ограничения традиционных и интеллектуальных инструментов при работе с программными репозиториями.
2. Спроектировать общую архитектуру системы автоматизированного построения окружений на основе LangGraph и парадигмы ReAct, определив структуру взаимодействия компонентов и организацию процесса выполнения.
3. Спроектировать модель выполнения ReAct-агента, включающую глобальное состояние, механизм маршрутизации, систему инструментов, обработку наблюдений и механизм восстановления состояния контейнера.
4. Реализовать программный прототип системы на языке Python с поддержкой Docker-контейнеров, автоматического анализа репозитория, запуска тестов, оркестрации выполнения и генерации Dockerfile.
5. Провести экспериментальное исследование системы на выборке GitHub-репозитория и оценить ее применимость для автоматического конфигурирования окружения и запуска тестов в изолированной среде.

1. Обзор современных технологий и решений

1.1. Анализ современных подходов

Методы автоматизации конфигурирования программных окружений можно условно разделить на две категории: статические, опирающиеся на интерпретацию метаданных и эвристические правила, и динамические, основанные на агентных системах с большими языковыми моделями, функционирующих в итеративном цикле «рассуждение — действие — наблюдение». В данном разделе последовательно анализируются принципы и структурные дефекты каждого из направлений.

1.1.1. Фундаментальные ограничения статических методов

В разработке на языке Python конфигурация вычислительных окружений традиционно основывается на декларативных файлах метаданных, таких как `requirements.txt` или `pyproject.toml`. По своей сути эти файлы представляют собой статический снимок зависимостей, поддержка которого зависит от субъективных суждений и ручного труда разработчика. В контексте открытых репозиторий (OSS) подобный подход, опирающийся на ручное сопровождение статических деклараций, обнаруживает два принципиально непреодолимых дефекта.

Во-первых, информационная полнота метаданных не может быть гарантирована. Файлы метаданных, как правило, фиксируют лишь зависимости верхнего уровня от Python-пакетов, полностью игнорируя системные зависимости и низкоуровневые C/C++ библиотеки, необходимые для корректного выполнения программ. Поскольку информация является неполной уже на входе, статический анализ, основанный на таких метаданных, принципиально неспособен выявлять ошибки конфигурации, связанные с системным окружением. Эмпирические исследования показывают, что около 68% [12] конфигурационных проблем не обнаруживаются только по метаданным — и эта цифра отражает не случайную небрежность инженерной практики, а прямую границу

внутренних возможностей статической парадигмы.

Во-вторых, сложность разрешения зависимостей превосходит обрабатывающую способность статических правил. Транзитивные зависимости (transitive dependencies) в экосистеме Python образуют многомерную сеть ограничений, в которой конфликт версий в любом узле способен вызвать глобальный сбой сборки. Ситуация дополнительно осложняется тем, что значительная часть Python-пакетов распространяется в форме C-расширений, процесс компиляции которых жёстко привязан к библиотечному окружению хост-системы, а эта информация не отражается ни в Python-метаданных, ни в выводимых статическими правилами заключениях. Именно это явление получило название «dependency hell» [13] — структурная проблема, неразрешимая в рамках статической парадигмы.

Методологическим продолжением опоры на статические метаданные выступают эвристические инструменты, такие как `repo2docker` и `pipreqs`. Они пытаются вывести зависимости из исходного кода с помощью предопределённых статических правил и формировать на этой основе конфигурационные файлы. Подобные методы просты и эффективны в идеальных условиях, однако по своей сути они представляют собой сопоставление с образцами на неполной информации: как только система сталкивается со сложным конфликтом зависимостей или отсутствующей системной библиотекой, её способность к выводу немедленно отказывает. Эти инструменты не взаимодействуют со средой, не воспринимают сигналы обратной связи и не располагают никакими механизмами исправления ошибок — неудача является не случайностью, а закономерным исходом данной парадигмы при выходе за граничные условия.

Таким образом, коренная проблема статических методов заключается в структурном рассогласовании между принципом их работы и природой решаемой задачи: фиксированный, предопределённый набор правил пытается охватить открытое, динамически изменяющееся пространство зависимостей. Подобное рассогласование не может быть преодолено локальными инженерными доработками и требует сдвига на уровне самой парадигмы.

1.1.2. Принципы и ключевые дефекты агентных методов

Стремясь преодолеть границы статической парадигмы, исследования последних лет обратились к агентным системам на базе больших языковых моделей (LLM). Центральная идея таких систем состоит в том, чтобы рассматривать построение окружения как динамический процесс принятия решений: модель, используя фреймворк ReAct (Reasoning + Acting)[11], непрерывно взаимодействует со средой в цикле «рассуждение — действие — наблюдение» — выполняет команду, считывает её вывод, анализирует сигналы ошибок и определяет следующий шаг[18]. Теоретически такая замкнутая структура наделяет систему способностью адаптироваться к неизвестным ошибкам.

Однако, несмотря на концептуальные преимущества, современные агентные методы при их применении к конкретной задаче автоматизированного построения окружений обнаруживают ряд серьёзных дефектов. Эти дефекты не являются второстепенными деталями инженерной реализации — они указывают на принципиальную неполноту самого замысла.

Первый дефект — слабоструктурированное представление состояния, по существу ненадёжное. В современных широко используемых агентных системах информация о «состоянии» полностью спрессована в линейно растущую историю текстового диалога. Для простых задач, требующих всего нескольких шагов, такое представление ещё может кое-как работать; однако для задач построения окружений с многочисленными зависимостями и сложными причинно-следственными связями между шагами контекстное окно стремительно расходует, и критически важные сигналы о состоянии системы — какие пакеты уже установлены, какие конфликты подтверждены, какие пути испробованы и признаны безуспешными — тонут в массе избыточного текста, в результате чего «память» модели, по существу, деградирует. Система, неспособная надёжно отслеживать собственное текущее состояние, не может претендовать на стабильное управление процессом.

Второй дефект — отсутствие детерминированных управляющих

структур, делающее поток решений хрупким. Цепочка «рассуждение — действие» в рамках ReAct по своей природе линейна: каждый следующий шаг свободно генерируется моделью на основе текущего контекста, без каких-либо заранее заданных ограничений на процесс. Это влечёт два прямых следствия. Во-первых, сталкиваясь с конфликтами зависимостей, требующими многоветвевое исследование, система неспособна к структурированному циклу «гипотеза — проверка — откат» и может лишь вслепую продвигаться по единственному пути или топтаться на месте. Во-вторых, когда результат выполнения команды требует кардинального изменения стратегии верхнего уровня — например, при обнаружении несовместимости критически важной библиотеки с текущей версией Python, — реакция системы целиком зависит от того, насколько качественно модель интерпретирует журнал ошибок. Возлагать ключевые решения, от которых зависит успех или провал всей сборки, на однократное рассуждение модели над текстом на естественном языке — это архитектурная уязвимость, а не проектное решение, направленное на обеспечение надёжности.

Третий дефект — отсутствие формализованного управления исполнением, делающее заикливание неизбежным. В отсутствие внешнего управляющего механизма агент, столкнувшись с трудноразрешимым конфликтом зависимостей, с высокой вероятностью попадает в петлю повторения одних и тех же ошибочных действий. Это не спекулятивный риск, а закономерный поведенческий паттерн, предопределённый свойствами линейной структуры рассуждений. Модель в условиях ограниченного контекста неспособна выйти за пределы локального когнитивного тупика, и без внешней структуры, принудительно задающей «сменить путь» или «признать данный путь бесперспективным и откатиться», повторение попыток является в математическом смысле детерминированным.

Для устранения перечисленных дефектов текущие исследования начинают двигаться в направлении формализации поведения агентов в виде ориентированных графов состояний[19], где процесс принятия решений выполняется в управляемой рамке, образованной дискретными

узлами (Nodes) и логическими рёбрами (Edges). С помощью фреймворков оркестрации графов, таких как LangGraph[10], операции управления потоком — ветвление, откат и циклы — выводятся из пространства свободно генерируемых моделью текстовых стратегий и возводятся в ранг базовых архитектурных примитивов системы, тем самым ограничивая произвольность LLM на инженерном уровне. Вместе с тем необходимо трезво осознавать, что данное направление пока находится на ранней стадии: вопросы гранулярности формального моделирования, выразительной способности стратегий управления и практической применимости в крупномасштабных сценариях далеки от систематического разрешения.

1.2. Docker как основа выполнения и управления средой

Технологии контейнеризации предоставляют стандартизированную среду исполнения для сложных программных систем, при этом Docker[14] является де-факто основным инструментом для построения воспроизводимых окружений. За счёт механизмов операционной системной виртуализации контейнеры обеспечивают изоляцию процессов при совместном использовании ядра хост-системы, что позволяет гарантировать согласованность поведения различных сред выполнения и делает их ключевой основой для воспроизводимой и стабильной сборки программных систем.

В задачах воспроизводимости ключевые сложности связаны не только с построением среды, но и с обеспечением её детерминированного выполнения. По сравнению с традиционными виртуальными машинами контейнеры характеризуются меньшими накладными расходами и более быстрым запуском, что делает их особенно подходящими для автоматизированных рабочих процессов, предполагающих частое создание и уничтожение окружений. В системах непрерывной интеграции и доставки (CI/CD) контейнеры широко применяются на этапах тестирования, сборки и развертывания, снижая влияние различий между

окружениями[15].

С системной точки зрения применение слоистой файловой системы (Layered Filesystem, UnionFS) декомпозирует процесс конфигурации среды в последовательность неизменяемых логических слоёв[16]. Такая архитектура преобразует процесс построения среды из неявного состояния в явно наблюдаемую и аудируемую последовательность операций, где каждый слой фиксирует определённое промежуточное состояние системы. Это создаёт структурированную основу для анализа и отслеживания процесса сборки.

В задачах автоматизированного построения окружений важной характеристикой, помимо воспроизводимости, является восстанавливаемость. Из-за конфликтов зависимостей, ошибок компиляции или отсутствия системных библиотек среда может перейти в неработоспособное состояние, и при отсутствии механизмов управления состоянием это приводит к загрязнению окружения и нарушению последующих этапов выполнения.

Современные методы, основанные на интеллектуальных агентах, частично используют механизмы восстановления. Например, Repo2Run[9] применяет стратегии отката на основе обратной связи выполнения и демонстрирует незначительное улучшение успешности сборки. Однако данный механизм носит преимущественно эвристический характер: логика управления основана на интерпретации неструктурированной обратной связи и не включает явного моделирования состояния или формализованного управления выполнением, что ограничивает его применимость в сложных сценариях отказов.

В отличие от этого, Docker предоставляет более прямой механизм управления состоянием на системном уровне. За счёт операций `commit` и `rollback` состояние окружения может быть восстановлено к любому предыдущему состоянию, что позволяет реализовать итеративный процесс исправления ошибок и экспериментирования. Такая версияционная модель превращает процесс построения среды из одноразовой операции в итеративно уточняемый процесс и создаёт основу для стратегий автоматизации, основанных на обратной связи.

Дополнительно Docker предоставляет программный интерфейс (Docker SDK)[17], позволяющий внешним системам управлять жизненным циклом контейнеров и получать потоковые данные выполнения. Эти данные формируют основу наблюдаемости среды и позволяют использовать результаты выполнения как сигналы для принятия решений.

В целом, контейнеризация обеспечивает единый базис для изолированного выполнения, восстановления состояния и наблюдаемого исполнения, создавая устойчивую инфраструктурную основу для последующих методов динамического принятия решений и замкнутых систем управления на базе интеллектуальных агентов.

1.3. Анализ существующих подходов

С точки зрения архитектуры исполнения существующие подходы к автоматизации построения и выполнения программных окружений можно разделить на две основные парадигмы: статические методы конфигурации и агентные методы, основанные на взаимодействии с внешней средой. Такое разбиение отражает эволюцию от декларативного описания зависимостей к динамическому принятию решений в процессе выполнения.

В таблице 1 приведено сопоставление ключевых характеристик рассматриваемых парадигм.

Таблица 1: Сравнительный анализ архитектурных парадигм автоматизации программных окружений

Критерий	Статические методы	Агентные методы (LLM)
Управление зависимостями	На основе метаданных и декларативных спецификаций	Динамическое, через взаимодействие со средой
Модель выполнения	Предопределённая	Адаптивная
Представление состояния	Практически отсутствует	Слабоструктурированное (текстовое)
Обратная связь	Отсутствует	Активное использование сигналов среды
Обработка ошибок	Внешняя или ручная	Эвристическая, нестабильная
Восстановление состояния	Не поддерживается	Частично реализуется через повторные попытки и рефлексии
Использование контейнеризации	Как изолированная среда выполнения	Как среда выполнения и источник сигналов среды
Воспроизводимость	Высокая при фиксированных версиях	Зависит от стратегии агента

Из приведённого сравнения видно, что рассматриваемые парадигмы принципиально различаются по способу обработки неопределённости, представлению состояния и механизмам управления выполнением. Статические методы обеспечивают простоту начальной конфигурации, однако практически не учитывают поведение системы во время исполнения. Агентные методы вводят замкнутый цикл взаимодействия со средой и адаптивное поведение, но их представление состояния остаётся неструктурированным, а восстановление после ошибок носит эвристический характер и не подкреплено формальными гарантиями.

Примерами статических методов являются такие инструменты, как

repo2docker и pipreqs; к агентным относятся системы Repo2Run, SWE-agent и OpenHands. При этом большинство современных решений используют контейнеризацию Docker в качестве универсальной среды исполнения и изоляции, однако воспроизводимость, адаптивность и управление состоянием реализуются в них разрозненными механизмами.

Таким образом, в существующих решениях отсутствует единый механизм, сочетающий явное структурированное представление состояния, формализованное управление выполнением и систематическую обработку сигналов обратной связи. Данный разрыв мотивирует переход к таким моделям исполнения, как графовые или автоматные представления, в которых ветвление, откат и повторное выполнение являются архитектурно заложенными элементами, а не эвристическими решениями, принимаемыми на основе неструктурированного текста.

1.4. Выводы

В данной главе систематически рассмотрены современные технические решения в области автоматизации построения и выполнения программных окружений. Проведённый анализ показывает, что традиционные статические методы, опирающиеся на метаданные, принципиально зависят от неполноты декларативной информации, неспособны воспринимать реальное поведение системы во время выполнения и обнаруживают границы своих возможностей при столкновении со сложными конфликтами зависимостей. Агентные методы, объединяющие большие языковые модели и фреймворк ReAct, вводят адаптивное использование обратной связи через цикл «рассуждение—действие—наблюдение», однако их представление состояния остаётся слабоструктурированным, а отсутствие детерминированных управляющих структур приводит к нестабильности управления состоянием, хрупкости потока решений и подверженности заикливанию.

В то же время технологии контейнеризации, представленные Docker, предоставляют единую инфраструктурную основу для указанных ме-

тодов: они обеспечивают изоляцию и воспроизводимость окружения, а также встроенные примитивы управления состоянием, такие как `commit/rollback`. Однако Docker сам по себе не образует законченного решения для разрешения зависимостей или восстановления после ошибок — он выступает лишь как низкоуровневая исполнительная платформа, доступная для вызова различными сборочными методами.

Таким образом, собственно методологическое сравнение имеет место между статической парадигмой конфигурации и парадигмой интеллектуальных агентов на базе LLM. Сравнение отчётливо демонстрирует, что ни одна из этих двух парадигм не способна по отдельности объединить воспроизводимость, адаптивность, управление состоянием и структурированное принятие решений в рамках единой модели исполнения — первая фиксирует конфигурацию в виде предварительной декларации, вторая вверяет принятие решений неструктурированной текстовой обратной связи и свободной генерации; в обеих архитектурах отсутствуют встроенные, нативные механизмы ветвления, отката и повторного выполнения.

Следовательно, в предметной области сохраняется выраженный структурный пробел: отсутствие формализованной исполнительной модели, интегрирующей управление состоянием, обработку обратной связи и динамическое принятие решений в единый контролируемый процесс[20]. Устранение этого пробела требует перехода к представлениям в форме ориентированных графов состояний или конечных автоматов. Исходя из этого, в настоящей работе предлагается использовать фреймворк LangGraph в качестве центрального технического решения — формализовать процесс построения окружения через структурированный граф, что превращает ветвление, откат и итеративное исправление ошибок в фундаментальные, архитектурно встроенные элементы системы, а не в эвристически-реактивные действия. Сформулированный вывод определяет направление последующих глав: далее последовательно рассматриваются архитектура системы на базе LangGraph, реализация конкретных узлов графа и их экспериментальная оценка.

2. Архитектура системы сборки

2.1. Цели проектирования системы

Основной целью разрабатываемой системы является автоматизированное преобразование произвольного GitHub-репозитория в воспроизводимое Docker-окружение. Для оркестрации процесса используется фреймворк LangGraph[10]. В качестве входных данных система принимает URL репозитория GitHub и выполняет последовательность операций анализа проекта, конфигурации зависимостей, проверки тестовой среды и генерации Dockerfile. Результатом работы системы является Dockerfile, позволяющий воспроизводить тестовое окружение исследуемого проекта.

Процесс построения окружения рассматривается как динамическая система с механизмом обратной связи. Система должна не только выполнять установку зависимостей, но и анализировать результаты выполнения команд, выявлять ошибки конфигурации и корректировать стратегию дальнейших действий.

При проектировании учитывается гетерогенность репозитория: различия в версиях Python, системных зависимостях, менеджерах пакетов и тестовых фреймворках. Это исключает возможность использования фиксированных сценариев конфигурации и требует адаптивного подхода к построению среды.

Дополнительной сложностью является неопределённость процесса установки зависимостей. Ошибки версий, отсутствие системных библиотек и сбои тестовых сред требуют динамического принятия решений на основе фактических результатов выполнения команд.

Система должна обеспечивать воспроизводимость результата: ключевые действия по настройке окружения фиксируются и используются при формировании Dockerfile, позволяющего повторно создать тестовую среду в стандартной Docker-инфраструктуре.

Также система должна поддерживать параллельную обработку репозитория при сохранении изоляции контейнеров и независимости состояний выполнения. Это важно для пакетной обработки набора проектов

и проведения экспериментальных запусков.

2.2. Общая архитектура системы

Архитектура системы построена по многоуровневому принципу и включает уровень представления, уровень выполнения, уровень оркестрации, уровень узлов графа, уровень инструментов и уровень внешних ресурсов.

Между уровнями используется принцип однонаправленных зависимостей: верхние уровни используют нижние, но не наоборот. Это снижает связанность компонентов и повышает модульность системы.

Следует отметить, что в данной главе архитектура рассматривается на уровне функциональных слоёв и потоков взаимодействия между ними. Конкретные механизмы хранения состояния, маршрутизации в графе, управления откатом среды и вызова инструментов подробно рассматриваются в следующей главе.

Общая архитектура представлена на рисунке 1.

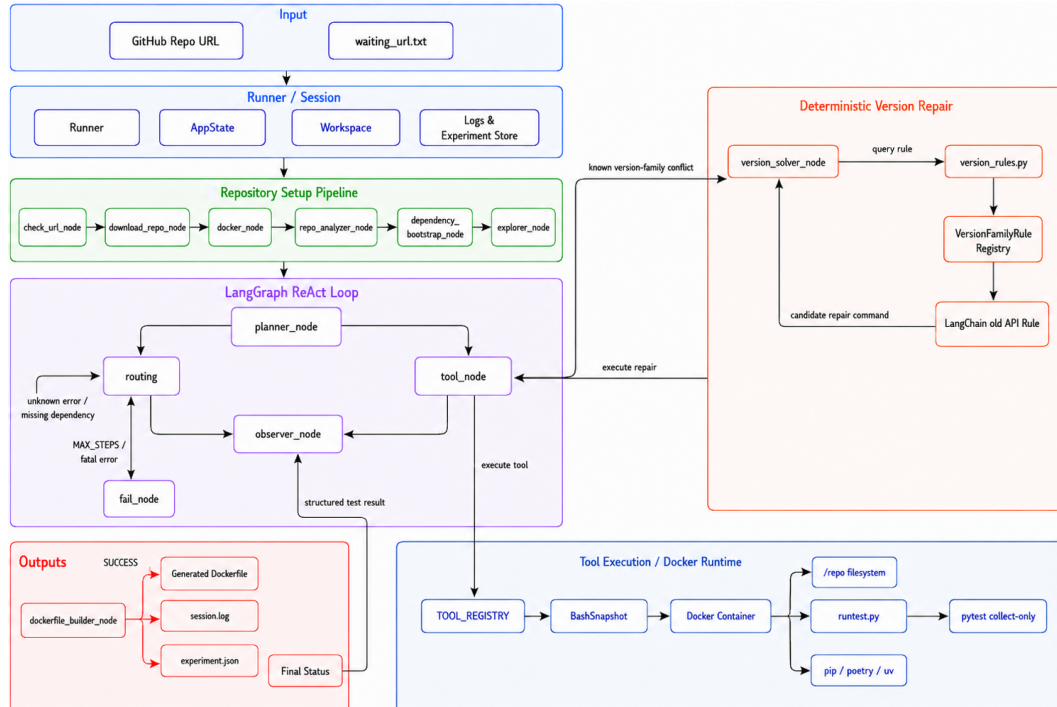


Рис. 1: Общая архитектура системы

Уровень представления отвечает за взаимодействие с пользователем

и поддерживает два режима работы: пакетный режим и API-сервис. Пакетный режим использует заранее подготовленный список URL-адресов репозиторий, например в виде входного файла со списком задач. API-режим предоставляет REST-интерфейс для интеграции с внешними системами.

Уровень выполнения реализует параллельную обработку задач с использованием пула потоков. Каждая задача выполняется в отдельном Docker-контейнере и изолированном рабочем пространстве. На уровне сессии поддерживаются объект общего состояния, рабочее пространство задачи, журналы выполнения и экспериментальные артефакты. Это обеспечивает изоляцию запусков, воспроизводимость результатов и удобство последующего анализа.

Уровень оркестрации реализован на основе LangGraph[10] и представляет процесс построения окружения в виде графа состояний. Объект общего состояния используется для передачи контекста между узлами графа и фиксации промежуточных результатов выполнения.

Уровень узлов графа разделяет процесс на три логические фазы: инициализация, итеративное выполнение и завершение. Инициализация включает проверку URL, получение рабочей копии репозитория, подготовку среды выполнения, первичный анализ структуры проекта и формирование начальной гипотезы о зависимостях. Итеративная часть реализует цикл ReAct, включающий планирование, выполнение и наблюдение. Завершение формирует итоговые артефакты выполнения, прежде всего воспроизводимое описание окружения в виде Dockerfile, а также сопутствующую диагностическую информацию.

Отдельно в архитектуре выделяется модуль детерминированного исправления конфликтов версий. Он взаимодействует с основным графом выполнения и применяется в тех случаях, когда проблема может быть сведена к известному классу несовместимостей зависимостей.

Уровень инструментов предоставляет агенту интерфейс взаимодействия со средой: выполнение shell-команд, запуск тестовой проверки, редактирование файлов и обращение к внешним источникам данных.

На нижнем уровне архитектуры инструменты не ограничиваются

абстрактными операциями, а взаимодействуют с реальной контейнерной средой выполнения, файловой системой проекта, механизмом запуска тестов и средствами установки зависимостей. Тем самым уровень инструментов связывает оркестрацию графа с Docker-runtime.

Уровень ресурсов включает систему контейнеризации, сервис языковой модели и внешние информационные источники, используемые для уточнения сведений о проекте и зависимостях. Контейнерная среда обеспечивает изоляцию выполнения, а внешние сервисы расширяют доступную агенту информацию о проекте и его зависимостях.

2.3. Структура графа выполнения

Процесс выполнения реализуется как ориентированный граф состояний, представленный на рисунке 2.

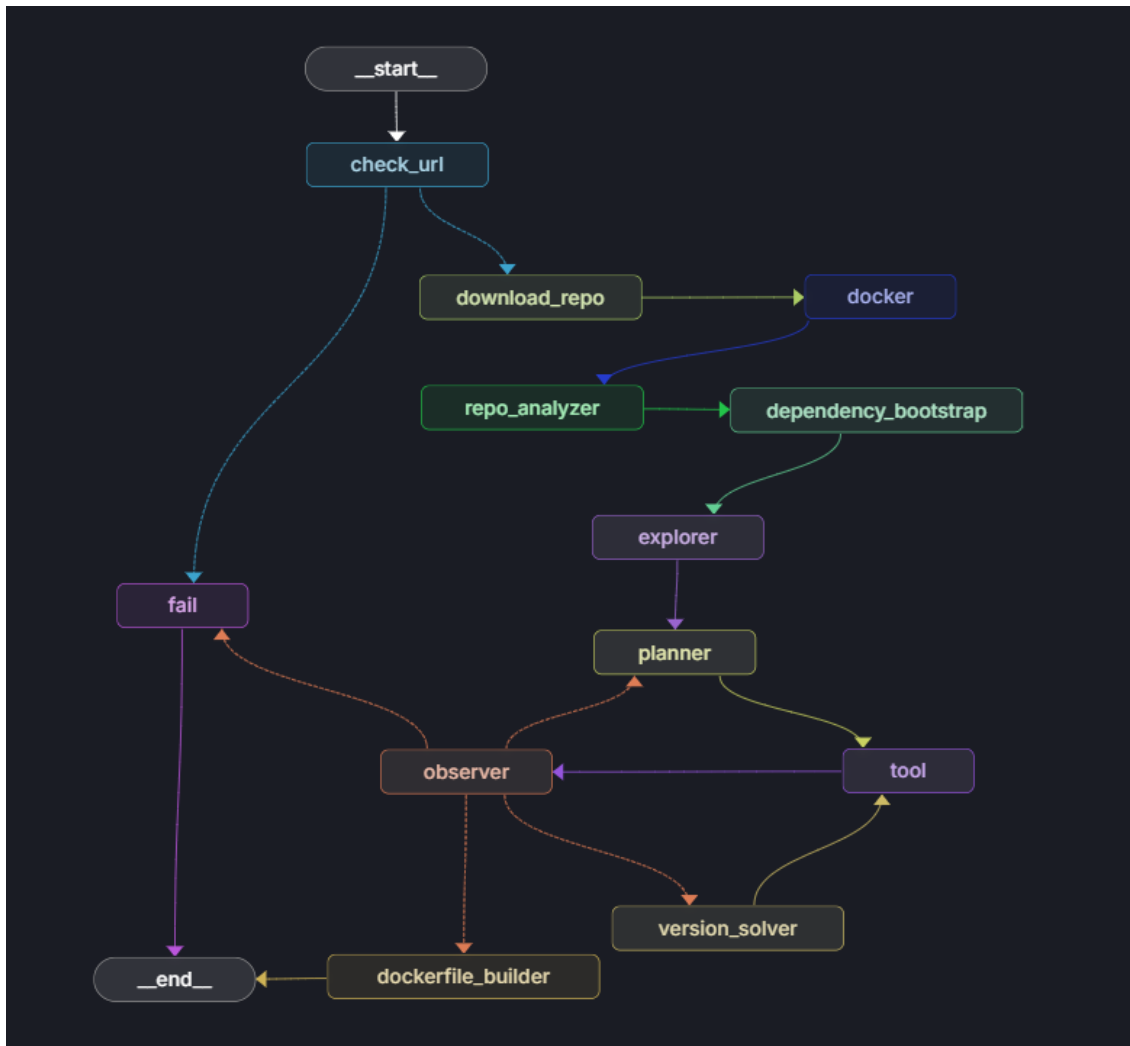


Рис. 2: Граф выполнения процесса построения окружения

Процесс разделён на три фазы.

Фаза инициализации включает подготовку среды: проверку URL, получение рабочей копии репозитория, подготовку контейнерной среды, первичный анализ структуры проекта и формирование начальной гипотезы о зависимостях [11].

Итеративная фаза реализует цикл ReAct, включающий планирование, выполнение действий и анализ результатов. Этот цикл повторяется до достижения верифицированного работоспособного состояния окружения или обнаружения критической ошибки.

Помимо основного ReAct-контура, архитектура предусматривает отдельную ветвь детерминированного исправления конфликтов версий. Она активируется при обнаружении известных несовместимостей зави-

симостей и затем возвращает управление в основной цикл выполнения.

Фаза завершения формирует итоговые артефакты выполнения, главным из которых является воспроизводимый Dockerfile. В случае неустраняемой ошибки система переходит в терминальное состояние fail.

С архитектурной точки зрения графовая модель позволяет описывать выполнение не как линейную последовательность команд, а как управляемую систему переходов между состояниями. Это особенно важно для задач конфигурации среды, где дальнейшее действие зависит от результата предыдущего шага, а ошибка не всегда означает немедленное завершение процесса.

2.4. Выбор технологий и архитектурных решений

Фреймворк LangGraph[10] выбран как основной механизм оркестрации, поскольку он поддерживает графовую модель выполнения, включающую циклы, условные переходы и хранение состояния. Это делает его более подходящим для задач, связанных с динамическим принятием решений, чем линейные цепочки выполнения.

Docker используется как механизм изоляции и управления средой выполнения. Каждый репозиторий обрабатывается в отдельном контейнере, что исключает влияние между задачами. Использование механизма снимков контейнерной среды позволяет сохранять промежуточные состояния окружения и выполнять откат при ошибках [14].

В качестве серверной части языковой модели используется API-совместимый сервис. Модель применяется для анализа проектов, генерации действий агента и формирования итоговых решений. Дополнительно реализован учёт токенов для оценки вычислительных затрат.

Архитектура принятия решений основана на парадигме ReAct, объединяющей рассуждение, действие и наблюдение в единый цикл обратной связи. Такой подход позволяет агенту не только генерировать команды, но и корректировать дальнейшие действия на основе результатов их выполнения.

Дополнительным архитектурным решением является выделение спе-

циального модуля для разрешения типовых версионных конфликтов. Такой модуль позволяет сочетать гибкость LLM-ориентированного цикла с преимуществами детерминированных правил в тех случаях, когда класс ошибки заранее известен.

Выбор перечисленных технологий обусловлен необходимостью совместить гибкость принятия решений, воспроизводимость вычислительной среды и возможность масштабирования системы на множество репозиторий с различной внутренней структурой.

2.5. Выводы по главе

В данной главе была представлена архитектура системы автоматизированного построения окружений. Определены цели проектирования, ограничения и ключевые требования, включая гетерогенность репозиторий, необходимость динамического принятия решений и обеспечение воспроизводимости результатов.

Предложенная архитектура объединяет графовую модель управления выполнением, контейнеризацию и интеллектуальное принятие решений на основе больших языковых моделей. Это позволяет реализовать автоматизированный процесс построения воспроизводимого окружения для Python-репозиторий.

Также выполнена декомпозиция системы на функциональные уровни и определены их взаимосвязи. Отдельно показано, что архитектура включает не только основной ReAct-контур, но и специальные механизмы предварительного формирования гипотезы о зависимостях, изоляции среды выполнения и детерминированного исправления типовых конфликтов версий.

Представленная архитектура формирует основу для реализации механизмов, описанных в следующей главе.

3. Модель и механизмы выполнения

В рамках фреймворка LangGraph выполнение системы основано на едином словаре состояния AppState, который используется для передачи информации между узлами графа. В отличие от архитектурного описания, представленного во второй главе, в данной главе рассматриваются проектные решения, лежащие в основе управления состоянием и организации процесса выполнения на концептуальном уровне.

В основе исполнительской модели лежит формальное представление выполнения как дискретного процесса принятия решений, где каждый шаг интерпретируется как переход между состояниями под воздействием выбранного действия и полученного наблюдения. Глобальное состояние системы определяется как:

$$S_{t+1} = \mathcal{T}(S_t, a_t) = (\mathcal{T}_E(E_t, a_t), H_t \oplus (a_t, O_t)) \quad (1)$$

где: S_t — состояние системы; a_t — действие агента; O_t — наблюдение среды; $\mathcal{T}_E$ — переход состояния среды; H_t — история взаимодействий.

Данная модель позволяет формализовать взаимодействие агента с окружением как управляемый процесс с обратной связью. На содержательном уровне состояние включает описание репозитория, профиль среды выполнения, историю действий и наблюдений, а также служебные признаки, определяющие дальнейшую маршрутизацию в графе. Тем самым AppState выступает не просто контейнером данных, а формальной моделью текущего этапа решения задачи.

С практической точки зрения использование единого состояния даёт три основных преимущества. Во-первых, оно обеспечивает согласованность взаимодействия между узлами графа: каждый узел принимает решение, опираясь на одно и то же представление контекста. Во-вторых, состояние позволяет сохранять историю рассуждений и действий, не разрывая связь между отдельными итерациями цикла. В-третьих, именно через состояние становится возможной маршрутизация между ветвями графа, поскольку критерии перехода формулируются как условия на поля AppState.

3.1. Формирование начального контекста

Цель этапа инициализации заключается в формировании минимально достаточного представления о репозитории до начала выполнения ReAct-цикла. Это позволяет Planner принимать обоснованные решения уже на первом шаге без дополнительных итераций анализа.

Процесс построения начального контекста можно рассматривать как две последовательные стадии статического анализа: анализ метаданных и конфигурации проекта, а затем анализ файловой структуры рабочей копии. Такое разделение обеспечивает переход от общей характеристики проекта к его более детальному внутреннему описанию.

3.1.1. Этап 1: анализ метаданных и конфигурации проекта

На первом этапе система формирует исходный профиль репозитория. В качестве источников контекста используются как внешние метаданные проекта, так и локально доступные конфигурационные файлы. К числу рассматриваемых признаков относятся основной язык разработки, общее назначение проекта, характер зависимостей, тип системы сборки и предполагаемый способ тестовой проверки.

Параллельно осуществляется анализ локальной копии репозитория. Система извлекает ключевые конфигурационные файлы, характерные для Python-проектов, включая `pyproject.toml`, `setup.py`, `requirements.txt`, `poetry.lock` и `uv.lock`. Эти файлы позволяют определить способ установки зависимостей, используемый менеджер пакетов, ограничения по версиям библиотек и возможные команды запуска тестовой проверки.

На основе полученных данных формируется структурированное представление проекта. Оно включает сведения о языке разработки (`project_language`), типе проекта (`project_type`), используемом тестовом фреймворке (`test_framework`), предполагаемой команде запуска тестов (`test_command`), команде установки зависимостей (`install_command`) и ключевых зависимостях (`key_dependencies`).

Важно подчеркнуть, что на данном этапе задача не состоит в точном

воспроизведении всего окружения проекта. Цель заключается в построении начальной гипотезы о том, как должен быть организован процесс установки зависимостей и последующей проверки. Поэтому результат первого этапа следует трактовать как семантический профиль проекта, достаточный для запуска итеративного цикла принятия решений.

3.1.2. Этап 2: анализ файловой структуры

На втором этапе выполняется анализ файловой структуры рабочей копии проекта. Его назначение состоит в уточнении представления о реальном содержимом репозитория: наличии тестов, распределении исходного кода, конфигурационных подкаталогов и артефактов, влияющих на дальнейшее поведение агента.

При сканировании исключаются технические директории, не относящиеся к логике приложения: `.git`, `__pycache__`, `node_modules`, `.venv`, `venv`, `.pytest_cache` и аналогичные им каталоги. Это позволяет избежать зашумления контекста и сосредоточить внимание на тех файлах, которые действительно значимы для понимания структуры проекта.

Результатом этапа является список относительных путей файлов, который объединяется с результатами первого этапа анализа. Благодаря этому формируется не только декларативное представление о проекте через конфигурационные файлы, но и операционное представление о его реальной организации на уровне файловой системы.

3.1.3. Формирование начального наблюдения

После анализа репозитория и сбора сведений о файловой структуре система формирует начальное наблюдение, которое добавляется в `agent_history` как первый элемент ReAct-цикла. Это наблюдение служит стартовым контекстом для узла Planner и позволяет агенту начинать работу не с пустого запроса, а с уже подготовленного описания проекта и среды выполнения.

В состав начального наблюдения включаются структура файлов, содержимое ключевых конфигурационных файлов, сведения о предпола-

гаемом типе проекта, обнаруженные зависимости, возможные команды установки и запуска проверки, а также параметры контейнерной среды.

[Н] [1] репозиторий R , контейнерная среда E начальное наблюдение O

$C \leftarrow$ извлечь ключевые конфигурационные файлы из R $F \leftarrow$ построить дерево файлов проекта $P \leftarrow$ определить тип проекта, зависимости и команды запуска

информация о проекте неполна $P \leftarrow$ дополнить профиль проекта с помощью эвристик

$O \leftarrow$ объединить F , C , P и параметры среды E добавить O в `agent_history` передать обновлённое состояние в Planner

Таким образом, начальное наблюдение выполняет роль первичной модели проекта и среды выполнения. Оно сокращает количество предварительных шагов анализа внутри ReAct-цикла и повышает информированность первого решения Planner.

3.2. Механизм ReAct-цикла выполнения

ReAct-цикл (Reasoning – Action – Observation) является ядром исполнительной модели системы. Цикл реализуется тремя логическими компонентами: `Planner`, `Tool` и `Observer`, которые образуют замкнутый контур и могут повторяться до достижения терминального состояния или исчерпания лимита итераций.

В содержательном отношении данный цикл решает три последовательные задачи. Сначала система формирует очередную гипотезу о полезном действии. Затем эта гипотеза материализуется через вызов инструмента. После этого результат действия интерпретируется как наблюдение, на основании которого принимается решение о продолжении или завершении процесса.

Таким образом, ReAct-цикл представляет собой не просто техническую схему вызова функций, а формализованный механизм итеративного приближения к корректной конфигурации среды.

3.2.1. Узел Planner: генерация действий

Узел Planner отвечает за генерацию следующего действия агента. На каждой итерации он получает доступ к текущему состоянию, включая профиль проекта, историю взаимодействий, ранее полученные наблюдения и общие ограничения среды выполнения.

Основная функция Planner состоит в том, чтобы преобразовать накопленный контекст в очередное осмысленное действие. Для этого он использует:

- сведения о типе проекта и предполагаемой системе зависимостей;
- информацию о тестовом фреймворке и целевой команде проверки;
- историю предыдущих действий и их результатов;
- структурированное начальное наблюдение, сформированное на этапе инициализации.

Результатом работы Planner является не произвольный текстовый ответ, а структурированное действие, пригодное для передачи в узел Tool. В этом смысле Planner решает задачу маршрутизации поиска: он должен определить, какой тип действия уместен на текущем этапе — исследование структуры проекта, установка зависимостей, запуск проверки или интерпретация обнаруженного сбоя через вспомогательные инструменты.

Отдельного внимания заслуживает управление длиной контекста. В процессе многократных итераций история взаимодействий может расти слишком быстро, что затрудняет последующую обработку. Поэтому на концептуальном уровне применяется стратегия контролируемого сокращения истории: сохраняются опорные наблюдения, необходимые для логической связности, тогда как второстепенные фрагменты постепенно вытесняются. Это позволяет сохранить непрерывность рассуждения, не перегружая модель избыточной информацией.

3.2.2. Узел Tool: выполнение инструментов

Узел Tool выполняет инструменты, выбранные Planner'ом. Содержательно он играет роль исполнительного интерфейса между рассуждением агента и внешней средой. Если Planner формирует гипотезу о следующем действии, то Tool материализует эту гипотезу в конкретный наблюдаемый результат.

На концептуальном уровне можно выделить два требования к инструментальному слою.

Во-первых, вызовы должны быть структурированными. Это означает, что действие агента представляется в виде явного указания типа инструмента и его аргументов, а не как неформальная текстовая инструкция. Такой подход делает переход от рассуждения к исполнению детерминированным.

Во-вторых, результат работы инструмента должен возвращаться в едином нормализованном виде. Независимо от того, выполнялась ли команда оболочки, запускалась ли тестовая проверка или осуществлялся анализ зависимостей, итогом должно быть наблюдение, пригодное для включения в историю взаимодействия и последующего анализа в Observer.

Наиболее важным инструментом в рамках рассматриваемой задачи является механизм выполнения команд в изолированной среде. Через него система получает возможность устанавливать зависимости, запускать тестовые процедуры, просматривать содержимое файлов и получать сигналы о состоянии окружения. Однако на концептуальном уровне важно не столько конкретное имя инструмента, сколько принцип: действие агента должно быть транслируемо в операцию над рабочей средой, а её результат — в структурированное наблюдение.

Кроме того, инструментальный слой выполняет и регулирующую функцию. Он ограничивает множество допустимых действий, не позволяя агенту выходить за рамки безопасного и интерпретируемого поведения. Тем самым Tool является одновременно исполнительным и контролирующим компонентом ReAct-цикла.

3.2.3. Узел Observer: маршрутизация состояний

Observer является управляющим ядром цикла. Его задача состоит в том, чтобы интерпретировать результат последнего действия и принять решение о дальнейшем направлении выполнения. В отличие от Planner, который формирует гипотезу, Observer опирается на уже полученный сигнал среды и выполняет функцию дискретной маршрутизации состояний.

Концептуально все результаты исполнения можно разделить на три класса: успешное продвижение, устранимое отклонение и критический отказ. В соответствии с этим выделяются три основных режима маршрутизации: **SUCCESS**, **RETRY** и **FAIL**.

Типовая семантика таких сигналов приведена в таблице 2.

Таблица 2: Типовая семантика сигналов завершения

Сигнал	Статус	Описание
Успешная проверка	SUCCESS	Среда настроена корректно, дальнейшая реконфигурация не требуется
Тесты не обнаружены	SUCCESS	Среда считается допустимой, хотя тестовые сценарии не найдены
Нет зависимостей	RETRY	Требуется дополнительная установка библиотек или системных пакетов
Конфликт версий	RETRY	Требуется согласование зависимостей или изменение конфигурации среды
Ошибка сборки	RETRY	Нужен дополнительный шаг исправления перед повторной проверкой
Критический сбой	FAIL	Продолжение цикла нецелесообразно без внешнего вмешательства

При **RETRY** в состояние добавляется новая инструкция для Planner, уточняющая характер проблемы и желательное направление следующего действия. Если же число последовательных повторов превышает допустимый предел, состояние переводится в режим отказа. Таким образом, Observer выполняет роль механизма стабилизации: он не только замыкает цикл обратной связи, но и предотвращает бесконечное повторение малорезультативных действий.

3.3. Управление поведением: системные промпты

Система использует три системных промпта, каждый из которых отвечает за свой уровень когнитивной организации процесса.

Промпт планировщика применяется в ReAct-цикле и задаёт роль агента, ограничения ответственности и общий порядок действий: анализ, установка, проверка, интерпретация результата. Его функция состоит в том, чтобы стабилизировать стратегию принятия решений и предотвратить переход к несогласованным или преждевременным действиям.

Промпт анализатора репозитория предназначен для преобразования структуры проекта в компактное структурированное описание. Он ориентирует модель не на генерацию рассуждений общего характера, а на извлечение признаков, необходимых для дальнейшего планирования: типа проекта, механизма установки, формата тестовой проверки и состава ключевых зависимостей.

Промпт итогового отчёта выполняет иную функцию: он формирует итоговое объяснение результата работы системы и помогает отнести возникшие проблемы к одному из классов — ошибки стратегии агента, ограничения самого проекта или сбои внешней среды исполнения.

Следовательно, система промптов распределяет когнитивную нагрузку между разными этапами обработки и тем самым делает поведение модели более структурированным и предсказуемым.

3.4. Механизм обеспечения устойчивости выполнения

Для защиты среды используется механизм снимков состояния контейнера. Его назначение состоит в том, чтобы локализовать побочные эффекты тех действий, которые могут существенно изменить конфигурацию окружения.

На концептуальном уровне все действия делятся на две группы:

- наблюдательные действия, не изменяющие состояние среды;

- модифицирующие действия, способные повлиять на состав зависимостей, файловую систему или параметры среды выполнения.

Перед выполнением модифицирующих действий создаётся снимок состояния контейнера. Если операция завершается неуспешно, система может восстановить предыдущее согласованное состояние и тем самым избежать накопления некорректных изменений. Это особенно важно в сценариях поэтапной установки зависимостей, когда ошибка на позднем шаге не должна разрушать уже достигнутый результат.

Содержательно данный механизм выполняет две функции. Во-первых, он повышает надёжность эксперимента, так как позволяет повторно запускать цикл в контролируемой среде. Во-вторых, он снижает стоимость неудачных гипотез: ошибочная установка трактуется как локальный сбой, а не как полная потеря рабочего состояния.

Дополнительным элементом устойчивости является блокировка явно опасных действий. Тем самым система защищается не только от случайных ошибок среды, но и от потенциально разрушительных действий, которые могли бы привести к необратимому нарушению состояния контейнера.

3.5. Механизм генерации Dockerfile

После завершения цикла система формирует воспроизводимое описание среды выполнения в виде Dockerfile. С теоретической точки зрения этот этап можно рассматривать как переход от динамически найденной конфигурации к её статическому представлению.

Основанием для построения Dockerfile служат:

- последовательность значимых действий, выполненных в процессе конфигурации;
- сведения о зависимостях, оказавшихся необходимыми для корректной верификации проекта;
- информация о базовой среде исполнения;

- результаты финальной проверки.

Итоговый Dockerfile должен не просто фиксировать набор установленных пакетов, но и воспроизводить логику построения среды: выбор базового образа, последовательность установки системных и прикладных зависимостей, а также контекст, необходимый для запуска проверки.

В более широком смысле генерация Dockerfile завершает цикл автоматизации: система не только находит рабочую конфигурацию, но и переводит её в форму, пригодную для повторного использования и переноса в другие вычислительные окружения.

3.6. Условия завершения выполнения

Система завершает работу при выполнении одного из нескольких условий.

Первое условие — достижение успешного состояния, при котором результаты проверки позволяют считать среду корректно настроенной. В этом случае основной цикл прекращается и инициируется формирование итогового артефакта.

Второе условие — исчерпание допустимого числа шагов или повторных попыток исправления ошибки. Данный режим защищает систему от бесконечного закливания и задаёт верхнюю границу вычислительных затрат.

Третье условие — возникновение неустранимого сбоя среды, при котором дальнейшее выполнение теряет смысл. К таким случаям относятся критические ошибки подготовки окружения, недоступность инфраструктуры выполнения или неинтерпретируемые результаты инструментов.

Следовательно, завершение процесса определяется не только фактом успеха или ошибки, но и общей политикой управляемого останова, ограничивающей цикл по времени, числу шагов и степени восстановимости среды.

3.7. Выводы по главе

В главе представлена модель выполнения системы на уровне управления состоянием, формирования контекста и организации цикла принятия решений. Показано, что AppState служит формальным представлением хода вычисления, а процесс решения задачи может быть интерпретирован как последовательность переходов между состояниями под действием агента и наблюдений среды.

Рассмотрены механизмы формирования начального контекста, включающего профиль репозитория, файловую структуру и параметры среды. Показано, что именно этот этап задаёт основу для осмысленного запуска ReAct-цикла и уменьшает необходимость предварительного исследования на первых шагах.

Проанализирован ReAct-цикл как центральный механизм выполнения, состоящий из узлов планирования, действия и наблюдения. Отдельное внимание уделено роли системных промптов, инструментального слоя, механизма снимков и принципам маршрутизации состояний.

Наконец, показано, что итогом работы системы является не только достижение корректной конфигурации среды, но и её фиксация в воспроизводимой форме, что делает модель пригодной для дальнейшего использования в задачах автоматизации сборки и настройки окружения.

4. Определение состояния и сборка графа

В четвертой главе рассматривается инженерная реализация агентной системы, предназначенной для анализа репозитория, настройки среды выполнения и построения воспроизводимого Dockerfile. Если в предыдущих главах были описаны концептуальные требования к интеллектуальному агенту, взаимодействующему с языковой моделью и внешней средой, то в настоящем разделе внимание сосредоточено на программной архитектуре, обеспечивающей выполнение этих требований на уровне кода.

В основе реализации лежит представление поведения агента как последовательности переходов между состояниями. Вместо набора разрозненных процедур система организована как вычислительный граф, в котором каждый узел отвечает за отдельный этап обработки: проверку входных данных, загрузку репозитория, подготовку контейнера, анализ проекта, планирование действий, вызов инструментов, интерпретацию результатов и завершение работы. Связующим элементом между узлами служит единая модель состояния приложения.

Особенностью данной реализации является то, что граф не ограничивается абстрактным ReAct-циклом в чистом виде. В проекте фактически используется расширенная схема, включающая подготовительный конвейер анализа репозитория, механизм автоматического восстановления зависимостей, ветвление для разрешения версионных конфликтов и подсистему построения итогового Dockerfile. Кроме того, в систему встроен механизм командных снимков контейнера, позволяющий безопасно выполнять потенциально рискованные операции установки зависимостей.

4.1. Модель состояния: структура AppState

Центральным элементом оркестрации является структура `AppState`, реализованная в проекте как типизированный `TypedDict`. Она хранит сериализуемое состояние вычисления и передаётся между узлами графа как основной носитель контекста. Такой подход обеспечивает детерминированность переходов, упрощает журналирование и делает поведение

системы воспроизводимым.

В отличие от абстрактной универсальной модели агентного состояния, в данном проекте **AppState** ориентирован на задачу автоматической конфигурации программного окружения для конкретного репозитория. Поэтому в нём содержатся не только поля, связанные с рассуждением агента, но и сведения о репозитории, контейнере, результатах тестовой верификации, зависимостях и попытках исправления ошибок.

Логически состояние можно разделить на несколько групп: идентификация сессии и репозитория, поля цикла ReAct, результаты анализа проекта, диагностическая информация о зависимостях и служебные признаки управления средой.

Таблица 3: Основные поля структуры состояния **AppState**

Поле	Тип данных	Назначение
<code>session_id</code>	string	Уникальный идентификатор сессии выполнения
<code>url</code>	string	URL репозитория GitHub, переданный пользователем
<code>is_valid</code>	boolean	Результат проверки корректности входного URL
<code>repo_path</code>	string	Путь к локально загруженной копии репозитория
<code>session_root</code>	string	Корневой каталог рабочей сессии
<code>container_id</code>	string	Идентификатор Docker-контейнера, используемого для выполнения
<code>python_version</code>	string	Целевая версия Python для контейнера
<code>step</code>	integer	Номер текущего шага ReAct-цикла
<code>agent_history</code>	list	История сообщений, наблюдений и инструкций, передаваемая модели
<code>llm_response</code>	string	Последний ответ языковой модели
<code>tool_calls</code>	list	Структурированный список вызовов инструментов, сгенерированных Planner
<code>last_output</code>	string	Последнее агрегированное наблюдение по результатам вызова инструментов
<code>runtest_result</code>	map	Структурированный результат инструмента тестовой верификации
<code>status</code>	string	Текущее логическое состояние исполнения
<code>feedback</code>	string	Краткая диагностическая обратная связь для следующего шага

Продолжение на следующей странице

Продолжение таблицы 3

Поле	Тип данных	Назначение
<code>parsed</code>	<code>map</code>	Разобранные признаки ошибок: отсутствующие модули, бинарные зависимости, конфликты версий и т.д.
<code>project_info</code>	<code>map</code>	Структурированное описание проекта, полученное на этапе анализа репозитория
<code>file_tree</code>	<code>string</code>	Снимок структуры файлов внутри контейнера
<code>token_total</code>	<code>integer</code>	Накопленное число токенов, использованных в сессии
<code>dependency_bootstrap</code>	<code>map</code>	Информация о сгенерированной базовой спецификации зависимостей
<code>version_solver_attempt</code>	<code>map</code>	Счётчик попыток автоматического разрешения версионных конфликтов
<code>install_attempted</code>	<code>boolean</code>	Признак того, что установка зависимостей уже предпринималась
<code>install_succeeded_since</code>	<code>boolean</code>	Признак успешной установки, ещё не подтверждённой повторным запуском верификации
<code>python_version_conflict</code>	<code>map</code>	Диагностическая информация о конфликте между версией Python и зависимостями
<code>import_diagnostics</code>	<code>map</code>	Результаты диагностики ошибок импорта

Помимо перечисленных данных, в состоянии присутствуют и служебные поля, управляющие поведением подсистемы восстановления среды. К ним относятся признаки отката последней команды, успешности восстановления снимка, инвалидации старого снимка после пересоздания контейнера, а также список автоматически созданных файлов `__init__.py`, используемых для исправления проблем импорта. Эти поля не являются частью высокоуровневой логики планирования, однако необходимы для корректной синхронизации состояния между инструментальным и оркестрационным слоями.

С инженерной точки зрения `AppState` удовлетворяет трём ключевым требованиям.

Во-первых, он сериализуем. В состоянии отсутствуют прямые ссылки на объекты Docker API, файловые дескрипторы и другие несериализуемые сущности. Это позволяет безопасно сохранять, журналировать и

анализировать состояние после завершения выполнения.

Во-вторых, состояние специализировано под предметную область. В отличие от общей модели диалогового агента, структура включает поля, отражающие специфику задачи конфигурации окружения: результат тестовой коллекции, классификацию зависимостей, диагностические данные об импортах и историю установочных команд.

В-третьих, состояние поддерживает частичное обновление. Каждый узел графа изменяет только ту часть `AppState`, за которую отвечает. Так, узел анализа репозитория обновляет `project_info`, узел выполнения инструментов обновляет `last_output`, `runtest_result` и служебные признаки, а узел `Observer` меняет `status` и `feedback`.

Важно отметить, что не все компоненты среды хранятся непосредственно в `AppState`. В проекте используется отдельный реестр времени выполнения, где по `session_id` хранятся экземпляры `DockerManager` и активный объект снимка среды. Такое разделение обусловлено тем, что объекты контейнера и снимков не являются сериализуемыми и должны существовать вне формальной модели состояния. Следовательно, `AppState` выступает как сериализуемая логическая модель вычисления, а реестр времени выполнения — как внешний слой управления инфраструктурой.

4.2. Инициализация вычисления и внешний контекст выполнения

Перед запуском основного цикла система формирует не только начальное состояние, но и инфраструктурный контекст сессии. Для каждой задачи создаётся отдельный идентификатор сессии, рабочий каталог, директория для временных файлов и каталог для журналов и итоговых артефактов. После этого регистрируется объект, управляющий Docker-контейнером.

Начальное состояние задаётся в виде словаря с базовыми значениями полей. В частности, URL репозитория, идентификатор сессии и рабочие пути известны сразу, а поля анализа проекта, истории агента и

диагностических результатов инициализируются пустыми значениями.

```
procedure InitializeState(session_id, url, repo_path, session_root):  
    state.session_id := session_id  
    state.url := url  
    state.repo_path := repo_path  
    state.session_root := session_root  
    state.is_valid := false  
    state.python_version := "3.10"  
    state.step := 0  
    state.agent_history := []  
    state.project_info := {}  
    state.runtest_result := null  
    state.status := ""  
    return state
```

Важной особенностью реализации является разделение между логическим состоянием и инфраструктурным контекстом. Контейнер и менеджер Docker регистрируются по идентификатору сессии в отдельном потокобезопасном реестре, а `AppState` хранит только их идентификаторы и логические признаки. Благодаря этому граф остаётся чистым с точки зрения сериализации, тогда как контейнер может безопасно переживать несколько итераций ReAct-цикла.

4.3. Архитектура графа выполнения

В проекте граф строится с помощью `StateGraph` и компилируется в исполняемую схему переходов. Фактическая архитектура включает не только центральный ReAct-цикл, но и подготовительные узлы анализа репозитория, а также специальные ветви автоматического разрешения ошибок.

Основной маршрут выполнения имеет следующий вид:

```
check_url
```

```
-> download_repo
-> docker
-> repo_analyzer
-> dependency_bootstrap
-> explorer
-> planner
-> tool
-> observer
-> {planner | version_solver | dockerfile_builder | fail}
```

Таким образом, ReAct-цикл в данной системе встроен в более широкий конвейер, который сначала подготавливает информированное контекстное представление проекта, а затем уже передаёт управление агенту.

4.3.1. Узел `check_url`

Первый узел проверяет корректность URL репозитория. На данном этапе выполняется синтаксическая валидация адреса GitHub без обращения к API платформы, что позволяет избежать избыточных сетевых запросов и ограничений по частоте вызовов.

Если URL не проходит валидацию, система немедленно записывает в состояние признак ошибки и переводит выполнение в терминальный узел отказа. Иначе происходит переход к загрузке репозитория.

4.3.2. Узел `download_repo`

После успешной валидации система загружает репозиторий в локальное рабочее пространство сессии. Этот шаг материализует объект анализа в файловой системе и делает возможным как локальное чтение конфигурационных файлов, так и последующее монтирование репозитория в контейнер.

Результатом узла является путь `repo_path`, который далее используется как для анализа проекта, так и для запуска контейнерной среды.

4.3.3. Узел `docker`

Следующий узел создаёт Docker-контейнер на основе базового Python-образа и монтирует в него локально загруженный репозиторий. Контейнер становится основной средой выполнения всех дальнейших инструментов, включая запуск команд оболочки и тестовой верификации.

Здесь важно подчеркнуть, что контейнер в проекте является долгоживущим объектом сессии. Он не уничтожается после каждого действия, а сохраняется между итерациями ReAct-цикла, что делает возможной инкрементальную настройку окружения.

4.3.4. Узел `repo_analyzer`

После подготовки контейнера запускается узел анализа репозитория. Его задача состоит в том, чтобы извлечь структурированную информацию о проекте до начала итеративного агентного цикла.

На данном этапе система:

- читает ключевые конфигурационные файлы, такие как `pyproject.toml`, `requirements.txt`, `setup.py`, `tox.ini`, `pytest.ini`, `README.md`;
- определяет язык проекта и его тип;
- извлекает предполагаемые команды установки зависимостей и запуска проверки;
- выделяет объявленные зависимости и ограничения по версии Python;
- формирует структуру `project_info`, используемую далее Planner-узлом.

Следовательно, Planner работает не вслепую, а на основе предварительно построенной модели проекта.

4.3.5. Узел `dependency_bootstrap`

Одной из важных проектных особенностей является наличие узла `dependency_bootstrap`. Он активируется после анализа проекта и предназначен для тех случаев, когда в репозитории присутствуют исходные Python-файлы, но отсутствуют явные или полные декларации зависимостей.

В этом случае система пытается построить базовую спецификацию зависимостей автоматически, используя статический анализ импортов и вспомогательные эвристики. Полученный список рассматривается не как окончательное решение, а как начальная точка для дальнейшего уточнения через цикл ошибок импорта и запуска тестовой верификации.

Таким образом, проект реализует не только реактивное исправление ошибок, но и предварительную эвристику начальной конфигурации.

4.3.6. Узел `explorer`

Узел `explorer` завершает подготовительную фазу перед запуском ReAct-цикла. Он собирает структуру файлов внутри контейнера, фиксирует актуальную версию Python в среде и формирует первое наблюдение, которое помещается в историю сообщений агента.

В результате именно с узла `explorer` формируется начальный контекст для Planner: дерево файлов, содержание ключевых конфигурационных файлов, сведения о типе проекта, тестовом фреймворке, команде установки и обнаруженных зависимостях.

Такое решение принципиально важно: в данной архитектуре агент начинает рассуждение не с пустого пользовательского запроса, а с насыщенного инженерного наблюдения о состоянии проекта и среды выполнения.

4.4. Центральный ReAct-цикл

После прохождения подготовительных узлов система переходит к основной фазе исполнения, реализованной как цикл `Planner -> Tool`

-> **Observer**. Именно этот контур отвечает за итеративное уточнение окружения, устранение ошибок конфигурации и достижение состояния, в котором проект может быть корректно собран и проверен.

В общем виде цикл можно представить следующим образом:

```
while not terminal(state):  
    state := Planner(state)  
    state := Tool(state)  
    state := Observer(state)
```

Однако в реальной реализации цикл дополнен:

- ограничением на максимальное число шагов;
- вспомогательной ветвью **version_solver** для детерминированного разрешения типовых версионных конфликтов;
- автоматическим повторным запуском верификации после успешных установочных команд;
- системой защит от бессмысленных повторов команд и заикливания поведения модели.

Таким образом, ReAct-цикл в проекте является не чистой теоретической схемой, а инженерно усиленной реализацией, ориентированной на устойчивую работу в реальной программной среде.

4.4.1. Узел **Planner**: генерация следующего действия

Узел **Planner** формирует очередной шаг агента на основе текущего состояния. Источниками контекста служат:

- структурированная информация о проекте **project_info**;
- история предыдущих действий и наблюдений **agent_history**;
- дерево файлов и выдержки из ключевых конфигурационных файлов;

- сведения о ранее предпринятых установках, диагностике импорта и конфликтах версий.

Важной особенностью реализации является компрессия истории перед очередным вызовом модели. Система сохраняет только значимые наблюдения: последнее состояние тестовой верификации, результаты диагностики импорта, сообщения Observer, последние высокоценные чтения файлов и небольшой хвост недавних сообщений. Это уменьшает объём контекста и одновременно удерживает критически важные опорные точки рассуждения.

Planner возвращает не произвольный текст, а структурированный список вызовов инструментов. Основной формат ответа модели представляет собой перечень `tool_calls`, например:

```
[
  {
    "name": "bash",
    "arguments": {
      "command": "pip install -r requirements.txt"
    }
  }
]
```

Если модель не сгенерировала корректный структурированный вызов, инструментальный узел пытается восстановить его эвристически из текста ответа. Такая двухуровневая схема повышает устойчивость к неполному соответствию формату.

4.4.2. Узел Tool: выполнение инструментов

Узел Tool отвечает за диспетчеризацию и исполнение вызовов, сформированных Planner. В проекте используется централизованный реестр инструментов, где каждому имени соответствует конкретный обработчик.

Инструмент	Назначение
<code>bash</code>	Выполнение команд внутри контейнера в каталоге <code>/repo</code>
<code>runtest</code>	Запуск тестовой проверки и сбор структурированной диагностики
<code>code_edit</code>	Применение diff-патчей к файлам проекта
<code>github_api</code>	Получение сведений о репозитории при необходимости
<code>pypi_info</code>	Запрос метаданных о пакетах Python из PyPI
<code>diagnose_import_tool</code>	Диагностика ошибок импорта и выбор способа исправления
<code>change_python_version</code>	Пересоздание контейнера с другой версией Python

Таблица 4: Инструменты, доступные агенту в текущей реализации

Результат каждого вызова преобразуется в нормализованное наблюдение, которое добавляется в `agent_history`. Для инструмента `bash` наблюдение содержит текст команды, её вывод и дополнительную информацию о том, была ли команда защищена снимком, была ли она заблокирована по соображениям безопасности и выполнялся ли откат среды.

Кроме того, узел Tool реализует несколько практических защит:

- блокировку опасных команд;
- запрет на повтор бессмысленных низкоценностных команд чтения;
- запрет на точный повтор уже неудавшейся установочной команды;
- принудительный запуск `runtest` после успешной установки зависимостей до выполнения новых действий.

Эти механизмы снижают риск того, что модель будет бесконечно повторять одни и те же действия без реального продвижения к цели.

4.4.3. Автоматическая верификация после установки

Одной из существенных проектных особенностей является связка между установочными командами и инструментом верификации. Если система распознаёт, что команда `bash` относится к установке или синхронизации зависимостей и завершилась успешно, она автоматически

инициирует повторный запуск `runtest`. Тем самым агент немедленно получает новую «опорную правду» о состоянии среды и не продолжает рассуждение на основе устаревших предположений.

Следовательно, в данной архитектуре `runtest` играет роль не просто очередного инструмента, а основного канала обратной связи от среды исполнения.

4.4.4. Узел Observer: интерпретация результата и маршрутизация

Узел Observer завершает очередную итерацию цикла и интерпретирует результат последней верификации. Ключевой принцип реализации состоит в том, что Observer доверяет прежде всего коду завершения `runtest`, а не эвристическому анализу текста лога. Это упрощает маршрутизацию и уменьшает вероятность логических ошибок.

В проекте используется следующая семантика кодов завершения:

Код	Статус	Интерпретация
0	SUCCESS	Коллекция тестов успешна, среда настроена корректно
5	SUCCESS	Тесты не обнаружены, но среда работоспособна
4	RETRY	Обнаружены отсутствующие Python-модули или системные бинарные зависимости
6	RETRY	Ошибка коллекции без окончательно классифицированной причины
7	RETRY	Зафиксирован версионный конфликт зависимостей
8	RETRY	Обнаружена ошибка сборки или компиляции
100	FAIL	Инструмент тестовой проверки недоступен, например отсутствует <code>pytest</code>
прочие	FAIL	Неклассифицированный отказ выполнения

Таблица 5: Правила интерпретации результатов `runtest` в узле Observer

В зависимости от результата Observer либо завершает выполнение, либо возвращает управление Planner, либо переводит граф в специальную ветвь разрешения версионного конфликта.

Фактическая логика маршрутизации после Observer такова:

- при `SUCCESS` происходит переход в узел построения `Dockerfile`;
- при превышении максимального числа шагов происходит переход в узел ошибки;
- при наличии правила для детерминированного разрешения конфликта выполняется переход в `version_solver`;
- во всех остальных случаях управление возвращается Planner.

Таким образом, Observer выполняет функцию конечного автомата принятия решений на основе верифицируемого сигнала от среды.

4.4.5. Узел `version_solver`

Ветвь `version_solver` представляет собой специальный узел, отсутствующий в абстрактной классической схеме ReAct, но критически важный для данного проекта. Он применяется тогда, когда система распознаёт типовой конфликт внутри семейства зависимостей и может предложить заранее определённую последовательность совместимых команд установки.

Вместо того чтобы снова передавать проблему модели в полностью свободной форме, узел выбирает очередную команду из набора детерминированных кандидатов. Это повышает воспроизводимость поведения системы в часто встречающихся сценариях конфликта пакетов.

Следовательно, архитектура проекта сочетает реактивное рассуждение модели с ограниченным набором жёстко заданных правил, что является типичным инженерным компромиссом между гибкостью и надёжностью.

4.5. Механизм снимков и отката контейнера

Наиболее чувствительной частью системы является выполнение команд, способных изменять программное окружение контейнера. К таким действиям относятся установка пакетов, синхронизация зависимостей, операции пакетных менеджеров и некоторые модифицирующие shell-команды. Для безопасной работы с ними в проекте реализован механизм `BashSnapshot` и `DockerSnapshot`.

4.5.1. Классификация команд

Перед выполнением команда классифицируется по трём основным категориям:

- опасные команды, которые блокируются немедленно;
- наблюдательные или низкорисковые команды, выполняемые без снимка;

- защищаемые команды, перед выполнением которых создаётся снимок состояния контейнера.

К опасным командам относятся, например, разрушительные операции вида `rm -rf /`, обращения к блочным устройствам, перезагрузка контейнера и иные действия, несущие угрозу целостности среды. Защищаемыми считаются прежде всего `pip install`, `poetry install`, `uv sync`, `apt-get install`, `npm install` и сходные операции.

4.5.2. Создание снимка

Если команда относится к защищаемым, перед её выполнением система сохраняет снимок контейнера через `docker commit`. При этом фиксируется не только файловая система контейнера, но и параметры, необходимые для его последующего пересоздания: имя образа, точки монтирования, режимы доступа, команда запуска и настройки GPU.

```
procedure ProtectedBashExec(command):  
    if command is dangerous:  
        return BLOCKED  
  
    if command is read_only:  
        return exec(command)  
  
    snapshot := docker_commit(current_container)  
    result := exec(command)  
  
    if result.exit_code != 0:  
        rollback(snapshot)  
  
    return result
```

Следует подчеркнуть, что снимок создаётся на уровне контейнера, а не на уровне логического состояния графа. Иначе говоря, откат относится к среде исполнения, а не к сериализуемому `AppState`.

4.5.3. Инкрементальный откат

Особенностью реализации является инкрементальная стратегия восстановления. После каждой успешной защищаемой команды снимок рассматривается как новая «хорошая» базовая точка. Если следующая установка завершится неуспешно, система откатится не к первоначально пустому контейнеру, а к последнему корректному состоянию, в котором предыдущие успешные шаги уже сохранены.

Это решение принципиально важно для агентного сценария, в котором настройка среды происходит поэтапно. Без инкрементального подхода любая локальная неудача приводила бы к потере всех ранее достигнутых результатов и к чрезмерному росту числа повторных действий.

4.5.4. Восстановление контейнера

При откате текущий контейнер останавливается и удаляется, после чего из сохранённого снимка создаётся новый контейнер с теми же параметрами запуска и монтирования. Далее ссылка на контейнер синхронизируется в реестре времени выполнения, чтобы последующие узлы графа продолжили работу уже с восстановленной средой.

Таким образом, откат не ограничивается логическим помечанием ошибки, а представляет собой полноценное пересоздание среды исполнения на основе сохранённого снимка.

4.6. Инструментальный слой

Инструментальный слой обеспечивает взаимодействие агента с внешней средой и служит нижним уровнем исполнения. В проекте он построен как реестр LLM-вызываемых инструментов с единым протоколом диспетчеризации.

4.6.1. Реестр инструментов

Реестр инструментов связывает символьное имя вызова с конкретной функцией-обработчиком. Благодаря этому Planner оперирует декларативными идентификаторами инструментов, а узел Tool выполняет их единообразно через общую точку входа.

Преимущества такого решения состоят в следующем:

- добавление нового инструмента не требует изменения логики Planner и Observer;
- централизованно контролируется набор доступных агенту действий;
- обеспечивается единый формат регистрации, диспетчеризации и журналирования вызовов.

4.6.2. Инструмент bash

Инструмент `bash` является основным средством взаимодействия со средой сборки. Внешне его интерфейс минимален: агент передаёт только строку команды, тогда как рабочий каталог фиксирован как `/repo`, а стратегия тайм-аута и необходимость снимка определяются внутренней логикой системы.

В отличие от абстрактного Bash-инструмента общего назначения, здесь:

- рабочая директория жёстко привязана к каталогу репозитория;
- тайм-аут выбирается автоматически в зависимости от класса команды;
- потенциально опасные действия блокируются;
- модифицирующие действия защищаются снимком контейнера;
- каждая команда журналируется и может быть позже использована для реконструкции Dockerfile.

4.6.3. Инструмент `runtest`

Инструмент `runtest` является ключевым каналом обратной связи от среды. Его задача состоит не только в запуске тестов, но и в формировании структурированной диагностики: числа собранных тестов, перечня отсутствующих модулей, бинарных библиотек, признаков конфликтов версий и ошибок сборки.

Именно данные `runtest_result` и `parsed`, а не текстовые логи в свободной форме, служат главным основанием для принятия решений в Observer. В этом смысле `runtest` выполняет роль формализованного сенсора среды.

4.6.4. Инструменты диагностики и редактирования

Инструмент `diagnose_import_tool` используется для классификации ошибок импорта и различения нескольких типов проблем: отсутствия внешнего пакета, локального модуля проекта, необходимости `editable-install`, проблем с `__init__.py` и других случаев. Это позволяет уменьшить количество ложных установок одноимённых пакетов из PyPI.

Инструмент `code_edit` отвечает за применение diff-патчей к файлам проекта. Он используется тогда, когда проблема не может быть решена одной лишь установкой зависимостей и требуется непосредственное изменение исходного кода или конфигурации.

Наконец, инструмент `change_python_version` позволяет пересоздать контейнер с другой версией Python, если конфликт обусловлен несовместимостью интерпретатора с диапазоном версий зависимостей.

4.7. Терминальные узлы и завершение работы

В текущей реализации граф содержит два фактических терминальных маршрута: успешное завершение через узел построения Dockerfile и неуспешное завершение через узел отказа.

4.7.1. Узел `dockerfile_builder`

Узел `dockerfile_builder` является основной успешной точкой завершения. В отличие от универсального агентного сценария, где финальным результатом может быть только текстовый ответ, в данной системе целевым артефактом является воспроизводимый `Dockerfile`, отражающий найденную конфигурацию окружения.

Для его построения узел использует:

- журнал выполненных действий;
- структурированный список команд, реально выполненных внутри контейнера;
- данные `pipdeptree` о фактически установленных версиях пакетов;
- сведения о выбранной версии Python и о внесённых правках.

Итогом работы становится файл `Dockerfile`, сохраняемый в выходной директории сессии.

4.7.2. Узел `fail`

Узел `fail` завершает выполнение при некорректном URL, превышении числа шагов либо неклассифицируемом отказе среды. Он формирует краткое итоговое описание причины неуспеха и сохраняет диагностическую информацию в артефактах сессии.

4.7.3. Вспомогательный узел `summary`

Следует отметить, что в кодовой базе присутствует также узел `summary`, предназначенный для генерации текстового резюме по журналам работы агента. Однако в текущей версии основного маршрута выполнения он не является обязательной частью успешной ветви графа. Поэтому с точки зрения фактической архитектуры проекта терминальным успешным узлом следует считать именно `dockerfile_builder`.

4.8. Выводы по разделу

В данном разделе была рассмотрена реальная архитектура программной реализации агентной системы. Показано, что управление поведением агента строится на сочетании сериализуемого состояния **AppState**, внешнего реестра инфраструктурных объектов и вычислительного графа, реализованного средствами **StateGraph**.

В отличие от абстрактной схемы общего агентного цикла, текущий проект использует расширенный граф, включающий узлы проверки URL, загрузки репозитория, создания контейнера, анализа проекта, генерации базовой спецификации зависимостей, исследования файловой структуры, центральный ReAct-контур, специализированную ветвь разрешения версионных конфликтов и узлы завершения.

Особое значение имеет тот факт, что инструментальный слой в данной системе не является пассивным исполнителем команд. Он реализует собственные механизмы безопасности, снимков, инкрементального отката, принудительной верификации после установки и защит от зацикливания модели. Благодаря этому достигается более высокая устойчивость к ошибкам и воспроизводимость процесса конфигурации окружения.

Таким образом, предложенная реализация демонстрирует, как концепция интеллектуального агента может быть преобразована в детализированную инженерную систему, пригодную для практической автоматизации настройки среды сборки и генерации воспроизводимого Dockerfile.

5. Экспериментальное исследование

5.1. Условия эксперимента и описание данных

Для оценки эффективности предложенной архитектуры было проведено экспериментальное исследование в контролируемой программно-аппаратной среде. Эксперименты выполнялись в стандартизированной контейнерной среде, что позволило снизить влияние особенностей хостовой операционной системы и обеспечить сопоставимость результатов.

Программно-аппаратная среда.

Экспериментальная апробация выполнялась на вычислительной системе со следующими характеристиками:

- процессор Intel Core i7-12700H (14 ядер);
- 32 ГБ оперативной памяти DDR5;
- хостовая операционная система: Windows 11;
- среда выполнения агента: Linux-среда внутри Docker-контейнера на базе образа `python:3.10`;
- ограничения контейнера: 4 vCPU и 8 ГБ RAM;
- модель DeepSeek-V4, используемая через API при фиксированном значении температуры ($T = 0$).

Таким образом, хостовая система использовалась для запуска инфраструктуры Docker, тогда как основные действия агента, включая установку зависимостей, выполнение команд и запуск тестовой проверки, выполнялись внутри изолированной Linux-среды контейнера. Использование контейнерной изоляции позволило минимизировать влияние внешних факторов и приблизить условия эксперимента к типичным сценариям CI/CD-инфраструктуры.

5.2. Описание набора данных

Для проведения исследования был сформирован набор из 98 публичных Python-репозиторий, размещённых на GitHub. Отбор выполнялся с учётом нескольких критериев:

- отсутствие готовых инструкций по запуску;
- наличие декларативных файлов зависимостей;
- возможность автоматизированной проверки корректности конфигурации.

Размер проектов варьировался от 1 000 до 35 000 строк кода. В выборку вошли проекты различного назначения, включая библиотеки машинного обучения, системы обработки данных и консольные утилиты.

Classification Summary		
Category	Description	Count
AI / LLM / Agent frameworks	LLM agents, AI frameworks, model tools	28
RAG / Vector / GraphRAG	RAG systems, retrieval, embedding	15
Web / API / FastAPI apps	Web services, API frameworks, backend templates	11
Dev tools / CLI / DevOps	Dev tools, CLI, automation	12
Data / ETL / Data tools	Data pipelines, data processing	9
Security / Test / Eval	Security, testing, evaluation frameworks	7
Scraping / Automation / Browser	Web scraping, automation	6
ML / Deep learning	ML tools, training frameworks	5
Database / DB tools	SQL / Graph DB tools	3
Other / Misc	Uncategorized	2
Total		98

Рис. 3: Классификация выбранных репозиторий по типу проектов

Использование разнородного набора репозиторий позволило оценить устойчивость системы в условиях различающихся зависимостей, структуры проектов и конфигурационных требований.

5.3. Исследовательские вопросы

Экспериментальное исследование было направлено на проверку нескольких ключевых аспектов работы системы.

RQ1 (эффективность): способна ли предложенная система повысить вероятность успешной автоматической конфигурации окружения по сравнению с существующими подходами?

RQ2 (затраты): каковы временные и вычислительные затраты системы при автоматической настройке проектов?

RQ3 (границы применимости): какие особенности программных проектов приводят к отказу автоматической конфигурации?

RQ4 (профиль инструментальных действий): какие типы инструментальных операций наиболее часто используются системой в процессе автоматической конфигурации?

5.4. Метрики оценки

Для количественной оценки эффективности использовались показатели, представленные в таблице 6.

Метрика	Описание	Связанный RQ
Success Rate (SR)	Процент успешно сконфигурированных сред	RQ1, RQ3
Rounds	Среднее число итераций взаимодействия с агентом (лимит: 30)	RQ2
Time-to-Success (TTS)	Время от начала работы до успешной верификации	RQ2
Command Distribution	Статистика частоты вызовов различных типов атомарных операций	RQ4

Таблица 6: Сводная таблица экспериментальных показателей

Выбранные метрики позволяют оценить как итоговую успешность системы, так и особенности её внутреннего поведения в процессе конфигурации среды.

5.5. Сводные результаты эксперимента

Общий коэффициент успешной конфигурации предложенной системы составил **91.8%**. Сравнение с Repo2Run, а также с базовыми подходами, приведёнными в работе [9], представлено в таблице 7.

Таблица 7: Сравнение эффективности различных подходов к конфигурации

Метод	Доля успешных конфигураций (%)
Proposed System	91.8% ↑
Repo2Run	83.87%
Pipreqs-based [9]	16.67%
LLM-generated [9]	22.1%

Значения для Proposed System и Repo2Run были получены в рамках проведённого экспериментального исследования на сформированном наборе из 98 репозиторийев. Результаты для Pipreqs-based и LLM-generated приведены по работе [9] и используются в качестве дополнительных ориентировочных базовых значений.

Полученные результаты показывают, что предложенная система достигает более высокого показателя успешной конфигурации по сравнению с Repo2Run в рамках проведённого эксперимента, а также существенно превосходит базовые подходы, представленные в работе [9].

Среднее время успешной конфигурации одной сессии составило **431.00 с**. Распределение временных затрат по этапам приведено в таблице 8.

Таблица 8: Распределение временных затрат по этапам выполнения

Этап	Среднее время (с)	Доля (%)
Install commands	276.70	64.2% ↑
LLM planning	90.08	20.9%
Run test	27.71	6.43%
Setup (clone+docker)	26.46	6.14%
Other	10.04	2.33%
Total	431.00	100%

Наибольшая доля времени выполнения приходится на команды установки зависимостей, которые занимают 64.2% общего времени сессии. Это связано с необходимостью загрузки и сборки пакетов, а также с повторными попытками разрешения конфликтов зависимостей. Вторым по затратности компонентом является планирование с использованием языковой модели (20.9%), включающее генерацию действий и анализ текущего состояния проекта. Выполнение тестовой верификации зани-

мает 6.43%, а подготовка среды, включая клонирование репозитория и создание Docker-контейнера, — 6.14%. Прочие операции составляют 2.33% общего времени выполнения.

5.6. Анализ результатов

5.6.1. RQ1: Эффективность конфигурации

Результаты эксперимента показывают, что предложенная система достигает высокого показателя успешной конфигурации. Как показано в таблице 7, разработанный подход достигает значения **91.8%**, что выше результата Repo2Run, полученного в рамках той же экспериментальной серии. Кроме того, этот показатель существенно превышает значения для Pipreqs-based и LLM-generated, приведённые в работе [9].

Такой результат может быть связан с использованием итеративного цикла взаимодействия с окружением. В отличие от статических методов, формирующих конфигурацию однократно, предложенная система анализирует ошибки выполнения и постепенно корректирует стратегию настройки на основе результатов проверки.

5.6.2. RQ2: Вычислительные затраты и потребление токенов

Вычислительные затраты системы оценивались через объём потребляемых токенов и число итераций агентного цикла, поскольку каждая итерация связана с обращениями к языковой модели и влияет на стоимость выполнения.

Экспериментальные результаты показывают, что потребление ресурсов зависит от сложности репозитория и количества необходимых шагов исправления окружения. Минимальное зафиксированное потребление составило **13.6k токенов**, среднее значение — **122.1k токенов**, а максимальное — **1167.8k токенов**.

При условной стоимости **20 рублей за 1 млн токенов** расчётная стоимость одной конфигурации составляет от **0.27 рубля** в минимальном случае до **23.36 рубля** в наиболее затратном случае. Средняя

стоимость конфигурации по потреблению токенов составляет приблизительно **2.44 рубля**. Эти значения учитывают только стоимость обращений к языковой модели и не включают локальные вычислительные ресурсы, используемые для запуска контейнера, установки зависимостей и выполнения тестов.

Распределение числа итераций, представленное на рисунке 4, показывает, что среднее число раундов агента составило **7**. Это указывает на то, что большинство проектов не требуют длительного многошагового взаимодействия с моделью, хотя отдельные сложные случаи могут приводить к значительно большему потреблению токенов.

Полученные результаты позволяют сделать вывод, что при выбранной тарифной оценке расходы на обращения к языковой модели остаются относительно низкими. При этом основной вклад в различие между простыми и сложными случаями вносит не постоянный диалог с моделью, а необходимость последовательного анализа ошибок среды, повторной установки зависимостей и уточнения конфигурации проекта.

Таблица 9: Оценка потребления токенов и расчётной стоимости конфигурации

Сценарий	Токены	Оценочная стоимость (руб.)
Minimum Consumption (Min)	13.6k	0.27
Average Consumption (Mean)	122.1k	2.44
Maximum Consumption (Max)	1167.8k	23.36



Рис. 4: Distribution of agent iteration rounds across 98 repositories (mean = 7)

5.6.3. RQ3: Границы применимости и причины неудач

Анализ журналов выполнения позволил выделить несколько факторов, которые могут ограничивать применимость системы автоматической конфигурации.

Одним из характерных факторов отказа являются сложные конфликты зависимостей. При установке пакетов система может сталкиваться с несовместимостью версий и расходовать значительную часть доступного бюджета итераций на попытки разрешения конфликтов.

Дополнительную проблему представляют неполные или нестабильные окружения, в которых отсутствуют отдельные библиотеки, плагины или вспомогательные компоненты, требуемые тестовым набором. В таких случаях возникают повторяющиеся ошибки импорта, что затрудняет достижение стабильного состояния среды.

Также наблюдаются случаи рассинхронизации между кодовой базой и тестами, когда структура пакетов или интерфейсы функций не соответствуют ожиданиям тестового набора. Подобные ошибки не всегда могут быть устранены исключительно за счёт управления зависимостями, поскольку они требуют изменения логики проекта или тестов.

Таким образом, отказы чаще связаны не с единичной критической ошибкой, а с накоплением нескольких конфигурационных проблем в

условиях ограниченного числа итераций. Это показывает, что предложенная система наиболее эффективна для проектов, в которых проблемы запуска связаны преимущественно с зависимостями, версией Python и настройкой окружения, но менее применима к случаям, требующим глубокого изменения исходного кода.

5.6.4. RQ4: Профиль инструментальных действий системы

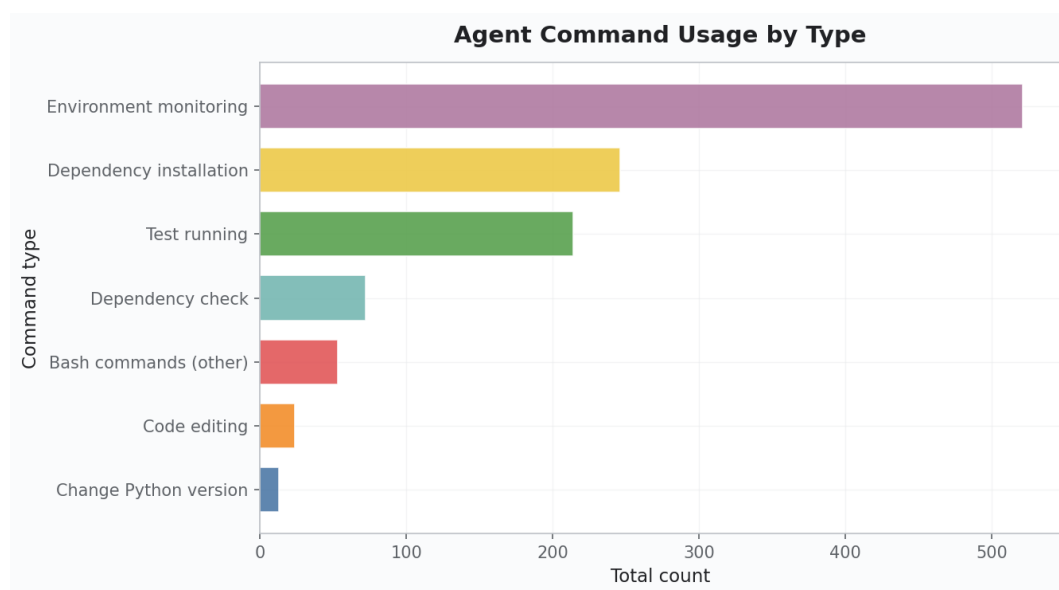


Рис. 5: Распределение типов команд, выполненных системой

На рисунке 5 представлено распределение типов действий, выполняемых системой в процессе автоматической конфигурации.

Полученные данные не являются полноценным ablation-анализом и не позволяют строго доказать причинный вклад каждого компонента в итоговый показатель успешности. Однако они позволяют охарактеризовать фактический профиль использования инструментов и показать, какие типы операций чаще всего задействуются системой при настройке окружения.

Анализ распределения команд показывает несколько характерных особенностей поведения системы.

Во-первых, заметная доля операций наблюдения и проверки указывает на то, что агент активно собирает информацию о состоянии проекта и окружения перед выполнением модифицирующих действий.

Это соответствует общей стратегии итеративной конфигурации, при которой каждое последующее действие выбирается на основе результатов предыдущих наблюдений.

Во-вторых, регулярные запуски тестовой проверки показывают, что система использует верификацию как основной источник обратной связи. После изменения состояния окружения агент получает новый сигнал о том, были ли устранены ошибки запуска или появились новые проблемы.

В-третьих, значительная доля операций установки и проверки зависимостей демонстрирует, что управление окружением является одной из наиболее трудоёмких частей автоматической настройки Python-проектов. Это также согласуется с временным анализом, где команды установки зависимостей занимают наибольшую долю общего времени выполнения.

Наконец, наличие команд смены версии Python и выполнения Bash-сценариев показывает, что система может реагировать на нестандартные инфраструктурные ситуации. Тем не менее такие действия следует рассматривать как часть наблюдаемого поведения системы, а не как отдельное доказательство эффективности каждого компонента без дополнительного сравнительного эксперимента.

Таким образом, результаты по RQ4 позволяют сделать осторожный вывод о том, что предложенная система использует преимущественно итеративный и диагностический подход к конфигурации среды, основанный на наблюдении, проверке гипотез и постепенной корректировке состояния окружения.

5.7. Выводы по главе

Проведённое экспериментальное исследование показало, что предложенная система достигает высокого показателя успешной конфигурации на наборе из 98 Python-репозиториях. Значение **91.8%** превышает результат Repo2Run, полученный в рамках той же экспериментальной серии, а также базовые значения Pipreqs-based и LLM-generated, приведённые в работе [9].

Анализ временных и вычислительных затрат показал, что среднее

время успешной конфигурации составляет **431.00 с**, а среднее потребление токенов — **122.1k**. При условной стоимости **20 рублей за 1 млн токенов** это соответствует примерно **2.44 рубля** на одну конфигурацию. Основная часть времени при этом приходится на установку и разрешение зависимостей.

Рассмотрение неудачных случаев показало, что основные ограничения системы связаны со сложными конфликтами зависимостей, неполными окружениями, ошибками импорта и несоответствием между кодовой базой и тестами. В целом результаты подтверждают практическую применимость предложенной архитектуры для автоматизации настройки окружения Python-проектов, особенно когда проблемы связаны с зависимостями, версией Python и конфигурацией среды выполнения.

Заключение

Проанализированы методы автоматизации создания программных окружений, выявлены следующие недостатки: плохое качество обнаружения неявных/отсутствующих зависимостей, слабая адаптивность к неоднородной структуре репозитория и тестовых фреймворков, ограниченное использование информации о сбоях времени выполнения, а также отсутствие механизмов автономного восстановления и повторной настройки окружения после ошибок. На этой основе обоснована целесообразность использования LLM-агента.

Спроектирована архитектура системы автоматизированного построения воспроизводимых программных окружений на основе LangGraph. В рамках архитектуры процесс обработки репозитория формализован в виде графа состояний, где глобальное состояние включает идентификатор сессии, параметры контейнерной среды, результаты анализа проекта и историю выполнения. Установлена многоуровневая организация системы, включающая уровень представления, уровень выполнения, уровень оркестрации, уровень графовых узлов, уровень инструментов и уровень внешних ресурсов. Установлены принципы взаимодействия компонентов, основанные на изоляции выполнения и однонаправленных зависимостях между уровнями.

Спроектирована модель выполнения ReAct-агента, основанная на формальном представлении процесса как дискретной системы принятия решений с обратной связью. Установлены ключевые компоненты исполнительного контура, включающие механизм формирования начального контекста, узел планирования действий, узел выполнения инструментов и узел анализа результатов. Формализованы правила перехода между состояниями системы на основе кодов завершения среды выполнения. Установлено, что использование итеративного цикла «анализ — действие — наблюдение — коррекция» обеспечивает адаптацию системы к неоднородным репозиториям и динамическим ошибкам конфигурации.

Реализован программный прототип системы на языке Python с использованием Docker и FastAPI. Установлена поддержка автоматиче-

ского анализа репозитория, выполнения тестов в изолированной среде, оркестрации процесса выполнения и генерации Dockerfile на основе результатов взаимодействия с окружением. Установлено, что использование контейнерной изоляции и автоматизированного управления зависимостями обеспечивает воспроизводимость процесса конфигурации и возможность его применения в CI/CD-сценариях.

Проведено экспериментальное исследование системы на выборке из 98 публичных Python-репозитория. Установлено, что показатель успешности автоматической конфигурации составил 92,8%, что существенно превышает результаты базовых подходов. Установлено, что среднее время выполнения одной задачи составляет около 670 секунд, а средняя стоимость обработки одного проекта составляет около 1,3 рубля. Выявлено, что основными ограничивающими факторами являются конфликты зависимостей, отсутствие системных библиотек и рассогласование между кодовой базой и тестовой инфраструктурой. Установлено, что применение итеративной стратегии ReAct оказывает ключевое влияние на эффективность автоматической настройки окружения.

Список литературы

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?," in Proc. Int. Conf. Learn. Represent. (ICLR), 2024. doi: 10.48550/arXiv.2310.06770.
- [2] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 37, 2024. doi: 10.48550/arXiv.2405.15793.
- [3] X. Wang et al., "OpenHands: An open platform for AI software developers as generalist agents," in Proc. Int. Conf. Learn. Represent. (ICLR), 2025. doi: 10.48550/arXiv.2407.16741.
- [4] GitHub, "GitHub Copilot — Your AI pair programmer." [Online]. Available: <https://github.com/features/copilot>.
- [5] Gao, C., Hu, X., Gao, S., Xia, X., Jin, Z. "The Current Challenges of Software Engineering in the Era of Large Language Models," ACM Trans. Softw. Eng. Methodol., vol. 34, no. 5, pp. 1–30, Jun. 2025, doi: 10.1145/3712005.
- [6] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang, "Training software engineering agents and verifiers with SWE-Gym," in Proc. 42nd Int. Conf. Mach. Learn. (ICML), PMLR, vol. 267, pp. 47717–47737, Jul. 2025. doi: 10.48550/arXiv.2412.21139.
- [7] J. Forde, T. Head, C. Holdgraf, Y. Panda, G. Nalvarte, M. Pacer, F. Perez, B. Ragan-Kelley, and E. Sundell, "Reproducible research environments with repo2docker," in Proc. Reproducibility in Mach. Learn. Workshop, Int. Conf. Mach. Learn. (ICML), 2018. [Online]. Available: <https://openreview.net/forum?id=B1lYOwuoxm> (accessed May 8, 2026).

- [8] bndr, pipreqs: Generate pip requirements.txt file based on imports of any project. (2015). [Online]. Available: <https://github.com/bndr/pipreqs> (accessed Nov. 2025).
- [9] R. Hu, C. Peng, X. Wang, J. Xu, and C. Gao, "Repo2Run: Automated building executable environment for code repository at scale," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2025. doi: 10.48550/arXiv.2502.13681.
- [10] LangChain, "LangGraph Documentation." [Online]. Available: <https://docs.langchain.com/oss/python/langgraph/overview> (accessed Apr. 2026).
- [11] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), 2023. doi: 10.48550/arXiv.2210.03629.
- [12] Y. Peng, R. Zhang, S. Liu, L. Gu, Y. Feng, and B. Sun, "Less is More? An Empirical Study on Configuration Issues in Python PyPI Ecosystem," in Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE), 2024. doi: 10.48550/arXiv.2310.12598.
- [13] X. Lin, J. Li, S. Li, H. Wang, and B. Li, "An Empirical Study on Python Library Dependency and Conflict Issues," in Proc. IEEE 24th Int. Conf. Softw. Qual., Rel. Secur. (QRS), 2024.
- [14] Docker Inc., "Docker: Accelerated container application development." [Online]. Available: <https://www.docker.com>
- [15] Y. Zhang, B. Vasilescu, H. Wang, and V. Filkov, "One size does not fit all: An empirical study of containerized continuous deployment workflows," in Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), 2018. doi: 10.1145/3236024.3236033.
- [16] Docker Inc., "Understanding the image layers," Docker Documentation. [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/building-images/understanding-image-layers/>

- [17] Docker Inc., "Docker SDK for Python — Docker SDK for Python 7.1.0 documentation." [Online]. Available: <https://docker-py.readthedocs.io>
- [18] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large Language Model Connected with Massive APIs," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2023. doi: 10.48550/arXiv.2305.15334.
- [19] J. Wang and Z. Duan, "Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models," arXiv:2412.03801, 2024.
- [20] T. Ridnik, D. Kredo, and I. Friedman, "Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering," arXiv:2401.08500, 2024.